ion_gym

An independent, self-contained toolkit for ion-optics simulation

User Manual — second edition

Clowers Research Group

Washington State University

Second edition, 2026-10-17 (first edition 2026-09-04)

Contents

Contents 1
1. What this toolkit is 1
1.1 What it is not 1
1.2 The two ways to drive it 1
2. Quick start 1
2.1 Seeing the trajectories 1
3. The specification file 1
4. Geometry 1
4.1 Declared primitives 1
4.2 Importing STL meshes 1
4.3 File organisation 1
4.4 Symmetry and mirror planes 1
5. Solving the field 1
5.1 Per-electrode bases and superposition 1
5.2 The iterative solvers 1
6. From potential to field 1
6.1 The two methods 1
6.2 When each is appropriate 1
7. Flying ions 1
7.1 The integrator 1
7.2 Time-varying drives 1
7.3 Collisions with background gas 1
7.4 Units 1
8. The pseudopotential view 1
9. Reading results 1
10. Limitations 1
11. Dashboard reference (tab by tab) 1
Ion Source 1
Voltages 1
Physics 1
Display 1
Impact Analysis 1
Geometry Import 1
Status 1
Multi FA 1
Config 1
View (plot pane) 1

Analysis (plot pane) 1
Thermal (plot pane) 1
PE Surface (plot pane) 1
Field Slice (plot pane) 1
Raster (plot pane) 1
Cache (plot pane) 1
Worked example captures 1
12. References 1
Appendix A. The instrument specification, shape by shape 1
A.1 The skeleton 1
A.2 The shape primitives 1
A.3 Signed frames: origin_mm 1
A.4 A symmetry example 1

1. What this toolkit is

ion_gym computes electrostatic, electrodynamic, and radio-frequency fields for ion optics simulations. It is a Python toolkit: geometry, voltages, drive waveforms, gas conditions and the ion source are all declared in a JSON specification file, solved with its own field solver, and integrated with its own trajectory kernel. It does not require, wrap, or call any external ion-optics package.

The design goal is simplicity and reproducibility. A specification file plus its geometry resources is a complete, portable description of a simulation: the same file gives the same field and the same trajectories on another machine. Every quantity the solver uses is declared somewhere you can read, rather than being set in a graphical interface and lost.

If you are learning the physics as well as the tool, start with the notebooks that ship beside this manual: 00–11 build from the Laplace equation to a complete multi-stage instrument, each runnable end to end, and V01–V06 validate the toolkit's physics against known answers — field solutions against analytic cases, Mathieu stability, thermalization, transport. This manual is the reference those notebooks lean on; they are the course.

1.1 What it is not

Being explicit about the boundaries is more useful than a feature list:

- It is not a general electromagnetic solver. It solves Laplace's equation for electrostatics on a uniform Cartesian or cylindrical grid. ***There are no magnetic fields, no space charge, and no dielectrics.***
- It is not a replacement for established commercial ion-optics packages. Where the two disagree, that disagreement is a finding to be investigated, not a result.
- It is not a CAD package. Though a shape editor exists as an ancillary control (section 13.1), geometry is generally built from declared primitives (e.g., circles, rectangles, and polygons) or imported as STL meshes produced elsewhere.
- It does not model gas dynamics. Collisions use a statistical model at a declared pressure and temperature; a uniform gas-flow vector can be declared in the specification and is honored on every route, but there is no computed flow field (section 7.3).

1.2 The two ways to drive it

Everything below can be done either from the browser-based application or from a script. The application is a viewer and an editor for the same specification files the scripts consume; it holds no configuration of its own. ***Anything you can set in the interface is a field in the file.***

2. Quick start

2.1 Installation

The toolkit installs as a normal Python package from the repository root. The core package needs only the scientific stack; the browser application and the notebooks pull extra dependencies, grouped as extras:

```
python -m pip install -e .              # core only
python -m pip install -e ".[editor]"    # + the browser application
python -m pip install -e ".[notebooks]" # + the teaching notebooks
python -m pip install -e ".[all]"       # everything
```

The first run after installation is slower than every later one: the physics kernels compile on first use and are cached to disk, and field solves are banked so a repeated build is a cache hit. Nothing needs to be configured. The one thing written outside the repository is the solved-field cache, in ~/.ion_gym_cache by default; set the ION_GYM_CACHE environment variable to move it.

A minimal session loads a specification, builds the field, and flies ions:

```
from ion_gym.io.sim_spec import SimSpec
from ion_gym.physics.sim_build import build_run

spec = SimSpec.from_json('examples/einzel_round_r-z.json')
model, fly, col_names, births = build_run(spec, verbose=True)

traj, summary = fly(0)          # fly ion 0
print(col_names)                # what the columns of traj mean
print(summary['tof'], summary['kind'])
```

Load specifications by **path**, not by pasting their text. A specification loaded from a path records where it came from, so relative resources — an STL directory, a births file — resolve against the specification rather than against your current working directory. This is what makes a specification plus its resources portable.

The returned fly is a function: calling fly(i) flies ion i and returns its trajectory array and a summary dictionary. Building the model and flying ions are separate steps because the field is expensive and the flight is cheap; one solved field serves any number of ions. In the summary, tof is when the flight ended and kind is the numeric fate code saying how — section 9 lists the vocabulary (for this deck every ion ends on the declared bounding plane, kind 3).

2.2 Viewing the trajectories

A raw trajectory array is rarely what you want to look at as a table. However, to view the flight, hand the flown paths back to the framework's own renderer — the same path every figure in this manual took:

```
from ion_gym.viz.viz_core import scene_from_simspec, report

trajs, fates = [], []
for i in range(len(births)):
    tr, summ = fly(i)
    if tr is not None and len(tr):
        trajs.append(tr)
        fates.append(str(summ.get('kind', '')))

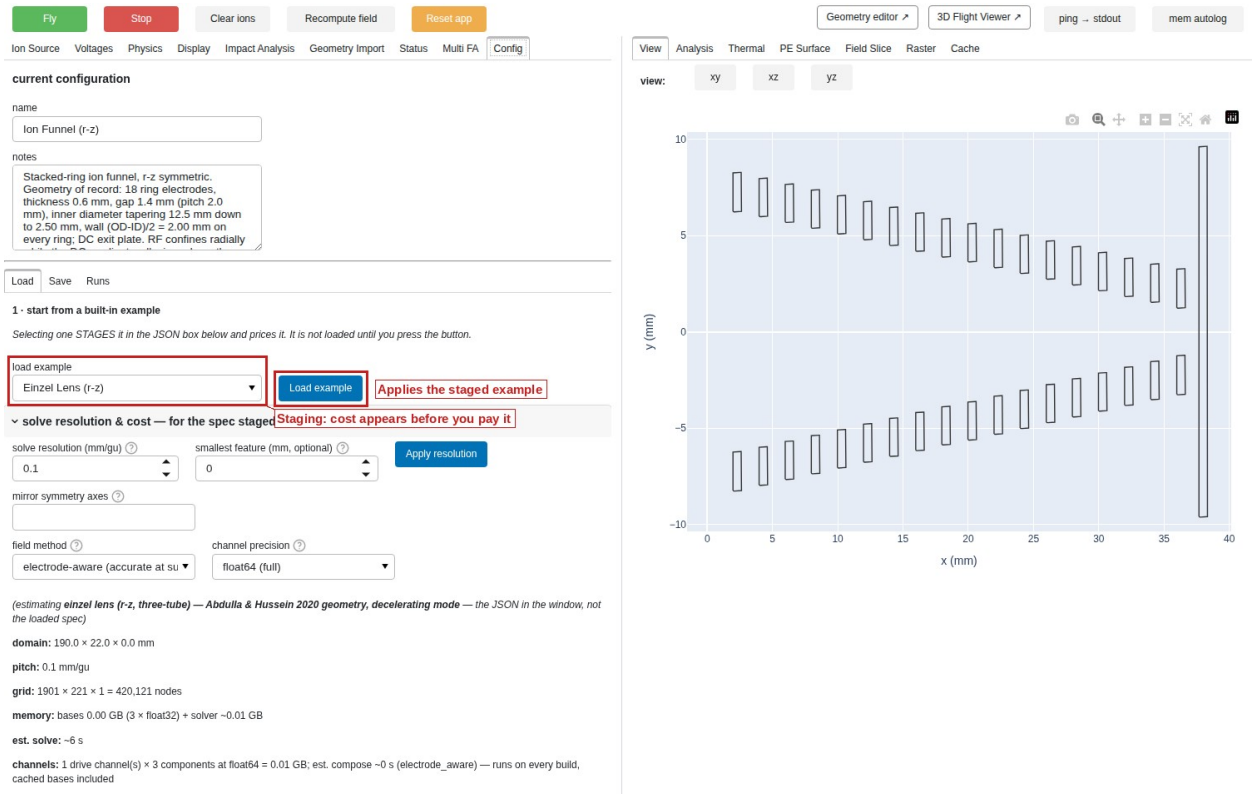
sc = scene_from_simspec(spec, model, field='phi',
                        trajs=trajs, fates=fates)
report([sc], 'outputs/einzel_quickstart', name='einzel quickstart')
```

The report draws the electrodes as solved, the equipotentials of the field the ions actually flew through, and every path colored by how its flight ended — and for a three-dimensional model it does so in each principal plane, because a single projection of a 3-D instrument hides exactly the off-axis behavior you are usually looking for.

2.3 The same flight in the application — a worked example

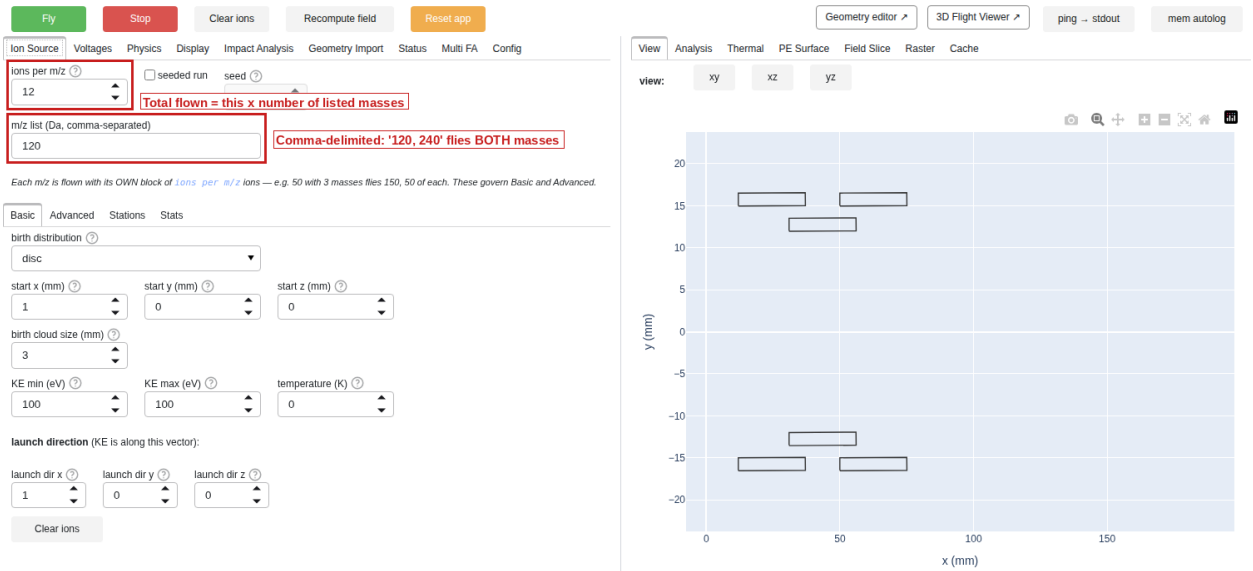
The application runs the identical loop with buttons instead of code. This walk uses the same shipped deck the script above loads — “Einzel Lens (r-z)” — and every screenshot below is a real capture of that session, one figure per step.

Step 1 — load the deck. Open the Config tab, pick “Einzel Lens (r-z)” in the example loader — picking stages the example and shows its solve cost before you commit — then press Load example.



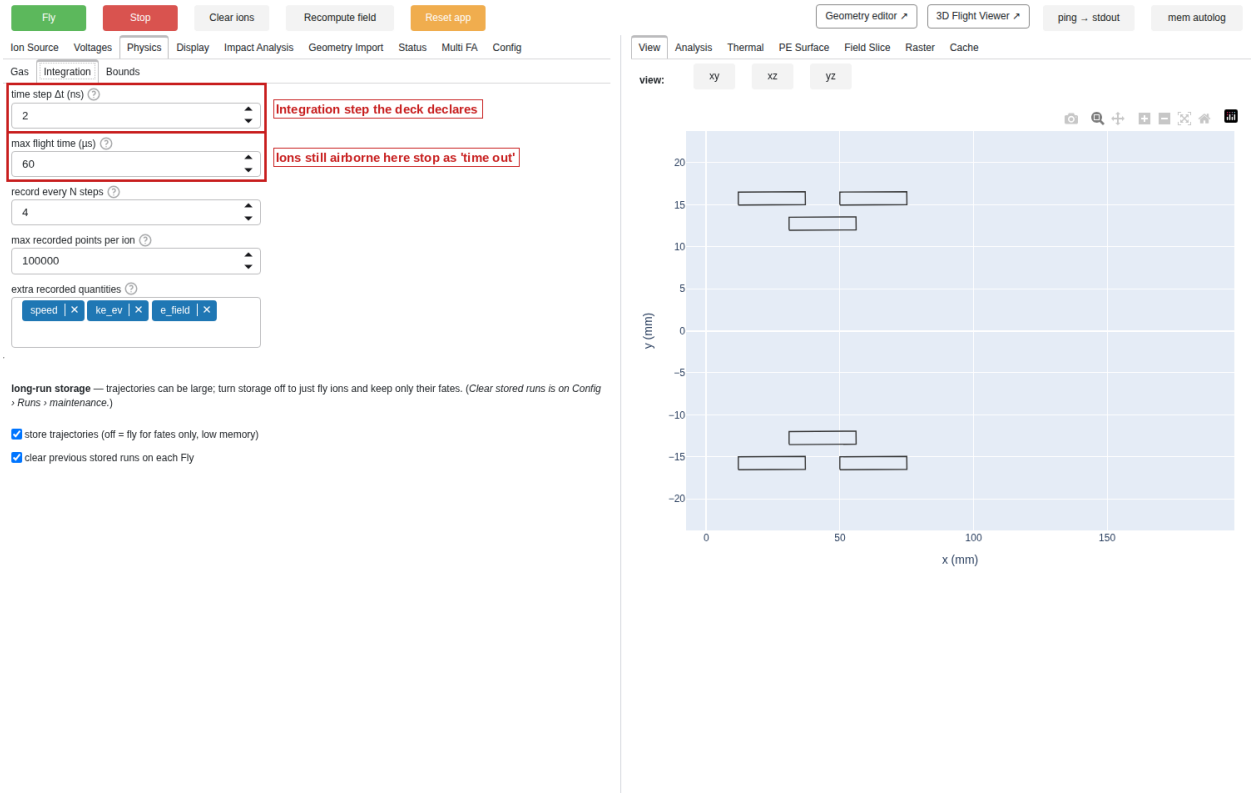
Step 1 -- Config tab: pick Einzel Lens (r-z) in the example loader. Selecting STAGES the example and shows its solve cost before you commit; the Load example button applies it.

Step 2 — check the ion source. The Ion Source tab now shows what the deck declares: 12 ions born on a 3 mm disc, 100 eV along +x, at *m/z* 120. The ion mass list is comma-delimited: “120” flies one mass; typing “120, 240” flies a full block of ions at each listed mass — the total flown is “ions per *m/z*” times the number of values in the list. The charge state is its own signed integer beside the list: the mass list carries masses in daltons, not mass-to-charge ratios.



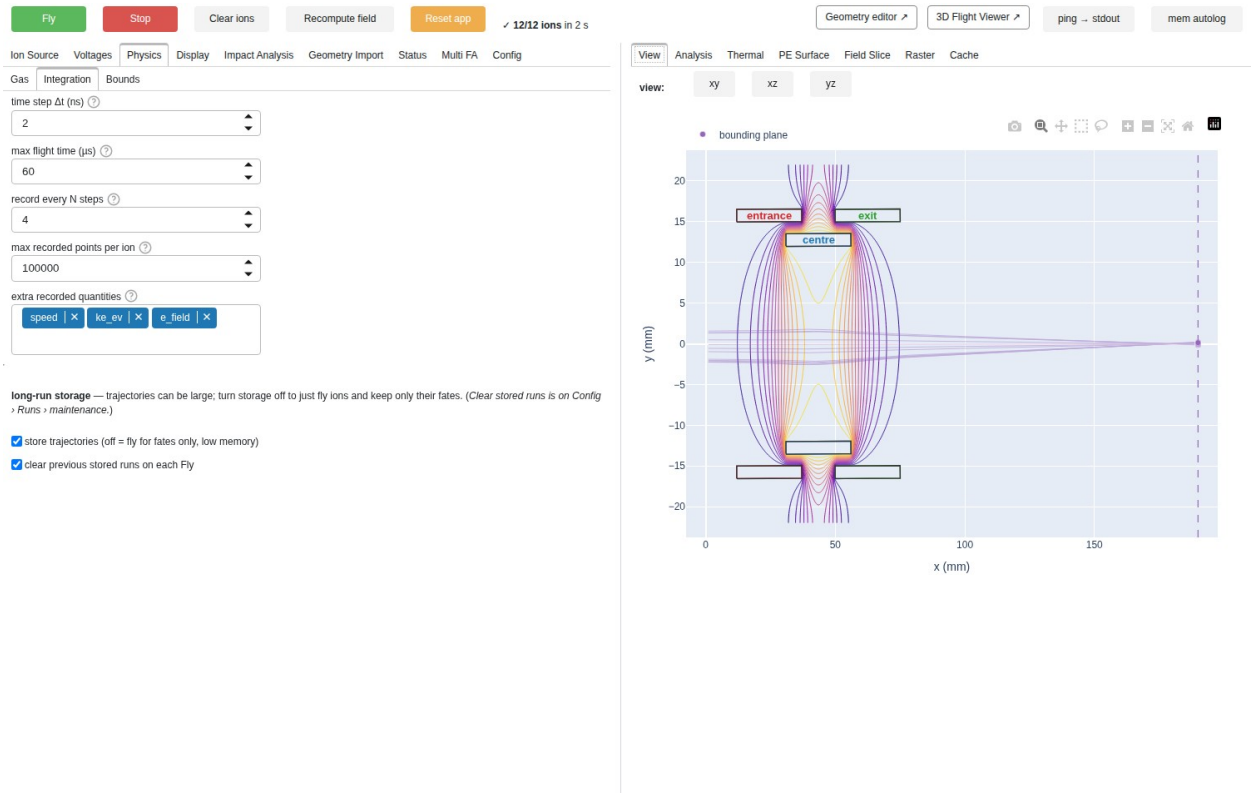
Step 2 -- Ion Source tab after loading Einzel Lens (r-z): the deck declares 12 ions on a 3 mm disc at 100 eV along +x, m/z 120. The m/z list is COMMA-DELIMITED: '120' flies one mass; '120, 240' flies a full block of ions at each listed mass.

Step 3 — check the flight conditions. The Physics tab carries the integration conditions the deck ships: a 2 ns time step, a 60 µs maximum flight time, and collisions off (vacuum). What this tab shows is what the integrator receives.



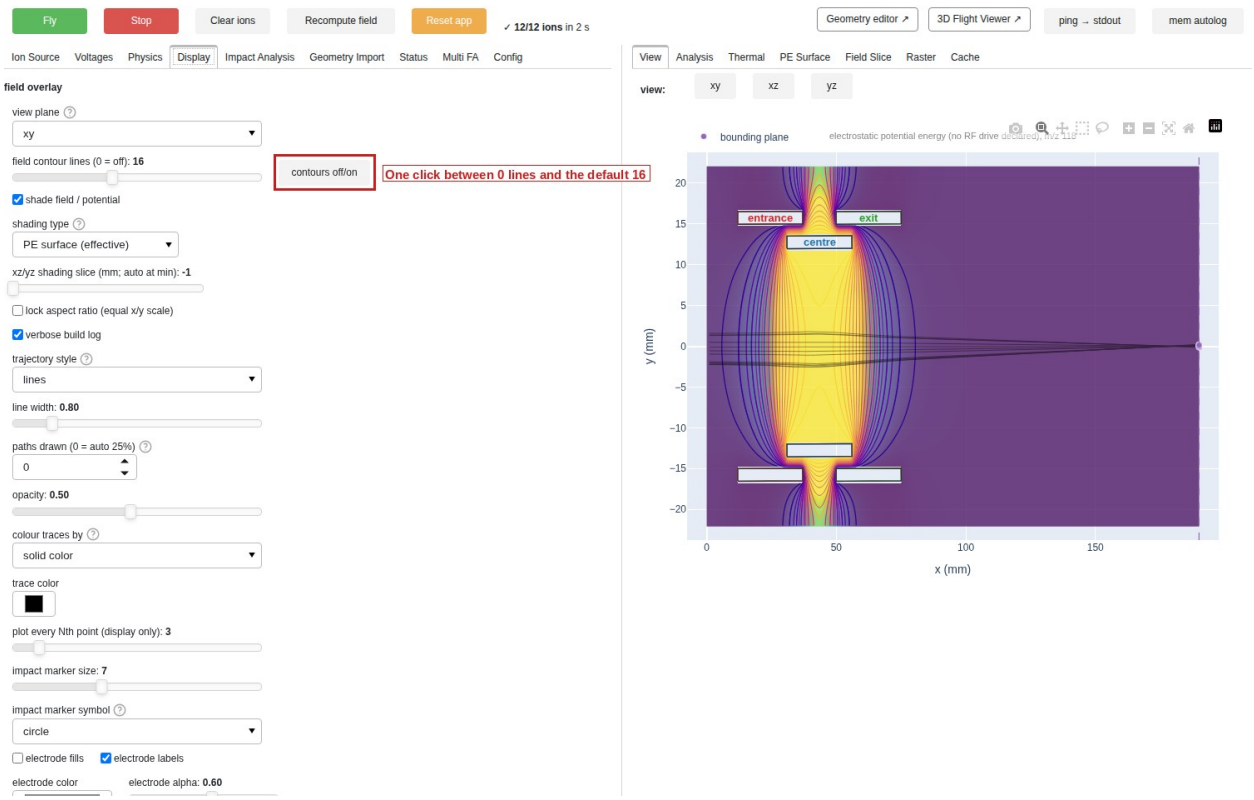
Step 3 -- check the flight conditions. The Physics tab's Integration sub-tab carries the conditions Einzel Lens (r-z) ships with: a 2 ns time step and a 60 µs flight-time ceiling. Collisions live on the Gas sub-tab beside it and are off for this deck (vacuum). What these fields show is what the integrator receives.

Step 4 — press Fly. The solve runs first (about ten seconds for this geometry; its progress prints to the Status tab), the ions fly, and the trajectories draw over the solved geometry in the View pane; xy / xz / yz projections are one click apart, and the 3D Flight Viewer opens the same paths in a rotatable scene.



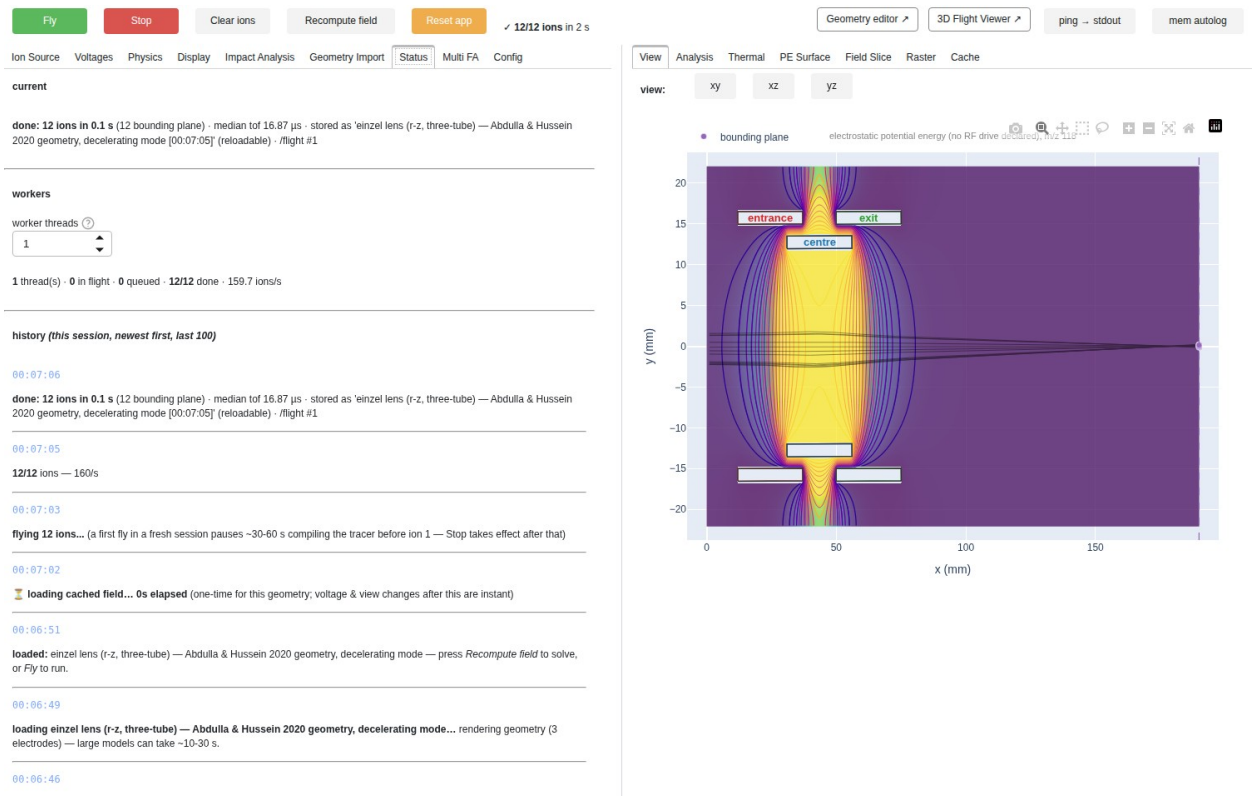
Step 4 -- press Fly: the trajectories draw over the solved geometry in the View pane (the one-button form of the section 2.1 script). xy / xz / yz projections are one click apart, and the 3D Flight Viewer opens the same paths in a rotatable scene.

Step 5 — show the field. On the Display tab, tick “shade field / potential”: the pane shades the field the ions actually flew through, with contour lines at the default count of 16. The contours off/on button beside the slider flips the lines with one click; the slider is the single value authority.



Step 5 -- the potential-energy landscape the ions flew through: Display tab, shading type 'PE surface (effective)', contour lines at the default 16. For this DC lens the PE surface is the electrostatic potential energy of the ion, so the einzel's three-tube focusing structure reads directly as the shape of the well; the contours off/on button flips the lines with one click.

Step 6 — read the result. The Status tab's run log names the solve, the fly, and the per-fate counts — the ground truth for what the flight did. For this deck expect all 12 ions to complete, every one ending on the bounding plane past the lens (the beam is focused through and exits the domain), with a median time of flight near 17 µs. The Analysis tab plots any recorded channel against any other for the same run (press Plot after choosing).



Step 6 -- the expected result, in the Status tab's run log: the solve, the fly, and the per-fate counts for the 12 ions. This log is the ground truth the chapter-2 prose quotes.

3. The specification file

A specification is JSON with a small number of top-level sections. The sections that matter most are:

Section	What it declares
geometry	Grid pitch, extents, symmetry, electrodes, their voltages and RF group membership, and how the field is differenced.
source	How many ions, their *m/z*, their spatial and velocity distribution, and the random seed.
integration	Time step, maximum flight time, and how often trajectory points are recorded.
collisions	Background gas species, pressure and temperature, or none for vacuum.
bounds	Optional planes that terminate a flight when crossed.
stations	Declared measurement planes in the flight: detectors that log or absorb, and plates with transmission windows (section 7.5).

Two properties of this file are worth internalizing. First, the resolution is declared, not inferred: mm_per_gu is the grid pitch in millimeters, and it sets both the accuracy and the cost of every solve. Second, defaults are omitted. A field left out of the file takes its documented default and is not written back, so two specifications that differ only in a default are byte-identical files. (Purely cosmetic fields — an electrode's display color, the recording caps — are reported once at load when left out; the decks that ship in examples/ state every field they rely on and load silently.)

4. Geometry

4.1 Declared primitives

The simplest geometries are built from declared shapes — boxes, cylinders, and their unions and differences — attached to named electrodes. Each electrode carries a name, an index, a DC voltage, and optionally the name of an RF group it belongs to. The name is not decoration: it is what the field maps, the legends and the analysis channels refer to.

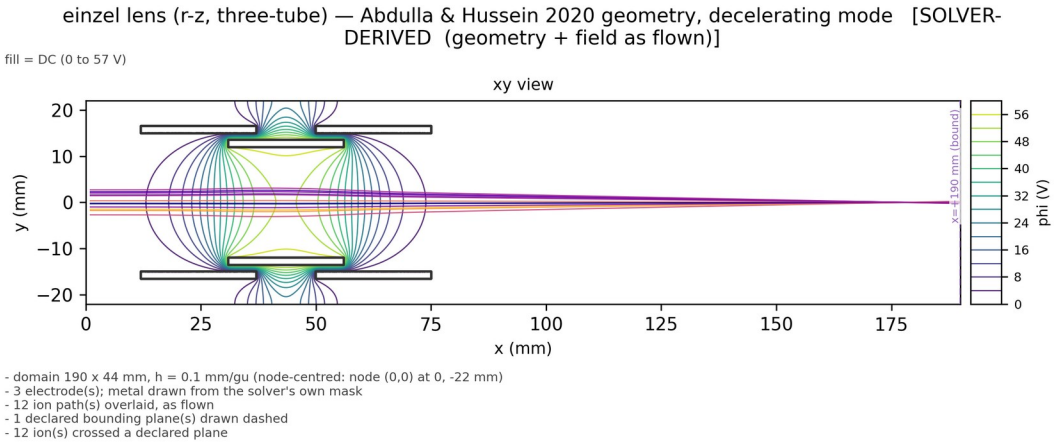


Figure 1. A round einzel lens declared as cylindrical (r-z) primitives. The shading is the solved potential; the outlined bodies are the electrodes exactly as the solver received them. The radial axis is drawn from −R to +R with the symmetry axis at zero.

4.2 Importing STL meshes

When a shape cannot be expressed as primitives, it is imported as an STL mesh — one file per electrode. This is the escape hatch for real hardware: draw the part in SolidWorks, Onshape or CadQuery, export STL, and point the specification at it.

The specification declares a directory with `stl_dir`, and each electrode names its own file. The directory is resolved relative to the specification file, so a specification and its meshes move together.

```
"geometry": {
  "mm_per_gu": 0.2,
  "stl_dir": "einzel_stl_plates",
  "electrodes": [
    {"index": 1, "name": "plate_1", "dc": 0.0,    "stl": "einzel3d_plate_1.stl"},
    {"index": 2, "name": "plate_2", "dc": −2000.0,"stl": "einzel3d_plate_2.stl"},
    {"index": 3, "name": "plate_3", "dc": 0.0,    "stl": "einzel3d_plate_3.stl"}
  ]
}
```

Meshes are voxelized onto the same uniform grid as everything else. A mesh does not buy you sub-grid accuracy: a curved surface is represented by the grid cells it occupies, so the solver pitch still governs how faithfully a curve or a thin feature is captured. A feature thinner than about two grid cells is not reliably resolved, and a mesh with holes or flipped normals may voxelize to something that is not what you drew. Preview the voxelization (Geometry Import tab) before trusting a mesh: what you see there is literally what the solver will solve.

Because the geometry is only as trustworthy as the files behind it, the toolkit records a SHA-256 hash of every STL a specification references. The preflight check compares the files on disk against those hashes and refuses to proceed on a mismatch, rather than silently solving a geometry that is no longer the one the specification describes.

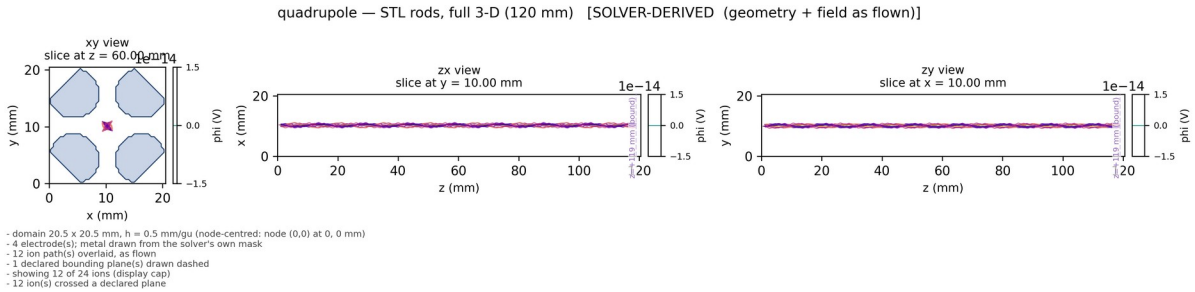


Figure 2. A quadrupole imported from STL rod meshes and solved as a full 3-D geometry, shown in three orthogonal projections. Any geometry with genuine depth is presented multi-axis: a single projection hides exactly the off-axis structure one needs to see.

4.3 File organization

A simulation is a directory, not a single file. The convention is:

```
my_instrument/
  instrument.json        <- the specification
  stl/                   <- meshes named by electrode
    plate_1.stl
    plate_2.stl
  births.csv             <- optional explicit ion start conditions
```

Keep the specification at the root of its own directory and reference resources with relative paths. Absolute paths make a specification unusable on any other machine, and are the single most common reason a shared simulation fails to load. Solved fields are cached separately, outside the specification directory, and are regenerated on demand — they are not part of what you share.

A project folder holding several devices, their meshes and custom birth files can grow complex, but one JSON file remains all that is needed to reproduce a result for a single instrument.

4.4 Symmetry and mirror planes

If a geometry is symmetric about a plane, only half of it need be stored and solved. A declared mirror axis halves the memory and roughly halves the solve time, and the solver reflects the appropriate boundary condition at that plane.

Symmetry is **declared, never assumed**. The toolkit will not detect symmetry for you, because a geometry that looks symmetric may not be symmetric in its voltages (e.g. a hexapole), and solving the wrong half is a silent error. A mirrored axis is displayed from −H to +H with the mirror plane at zero, so a coordinate you write in the specification is measured from the plane of symmetry and read back the same way.

5. Solving the field

Why a grid at all? Between the electrodes the potential obeys Laplace's equation, and for any realistic electrode shape that boundary-value problem has no closed-form solution — the geometry IS the problem. The classical response is to replace the continuous domain with a regular lattice of sample points, write the equation as a relation between each point and its neighbors (each node relaxes toward the average of its neighbors — the discrete statement that a harmonic function has no interior extrema), and iterate until the lattice stops changing. Everything downstream inherits the choice: the pitch of that

lattice is the resolution of the physics, which is why the specification declares it explicitly instead of inferring it.

5.1 Per-electrode bases and superposition

The field is not solved once for a set of voltages. Instead, Laplace's equation is solved once per electrode, with that electrode at a reference potential and every other electrode grounded. Each solution is stored normalized to unit electrode potential, giving a dimensionless response function — the fraction of that electrode's voltage seen at each point.

The field for any set of voltages is then the weighted sum of these bases. This has a consequence worth exploiting: changing a voltage is not a re-solve. Sweeping a lens voltage, retuning a stack, or optimizing a set of potentials re-weights bases that are already in memory, which is essentially free. Changing the geometry or the grid pitch is a re-solve, because the bases themselves depend on those.

Because a solve is expensive and a geometry is often reused, solved bases are cached on disk under a key derived from everything that affects them — the geometry, the pitch, the declared symmetry, and the mesh hashes. Any change to those produces a different key and a fresh solve.

5.2 The iterative solvers

Laplace's equation is discretized on the uniform grid and relaxed iteratively. The base method is successive over-relaxation (SOR), the classical accelerated relaxation scheme analyzed by Young **[3]**, applied in a red-black ordering so that each half-sweep is independent and can be executed in parallel. The relaxation factor is chosen from the grid dimensions rather than left to the user.

SOR alone converges slowly on large grids because it removes short-wavelength error (components of the residual that vary over a few grid cells) quickly and long-wavelength error (components that vary over a large fraction of the domain) very slowly. The multigrid solver addresses this by relaxing on a hierarchy of coarser grids, where the long-wavelength error is short-wavelength, and prolongating the result upward — the multi-level idea introduced by Brandt **[2]**. The implementation here is cascadic: it solves coarse-to-fine, using each level's solution as the next level's starting guess, and the final answer is produced by relaxing on the finest grid to the requested tolerance. This matters for a subtle reason — the fine-grid solution is the one the ions fly through, so it is solved directly rather than reconstructed from a correction.

Convergence is tested against a declared tolerance on the maximum change per sweep (watch the terminal console during a large solve to see this in action). If the solver reaches its iteration cap without meeting the tolerance it raises an error. It does not return the unconverged field. This is deliberate: an unconverged field that looks plausible is far more dangerous than a failed solve, because nothing downstream can detect it.

6. From potential to field

The solver produces a potential. The trajectory kernel needs an electric field, $E = -\nabla\varphi$, evaluated on the same grid. How that gradient is taken is a choice with real consequences, and it is a declared option.

6.1 The two methods

Plain gradient takes a central difference everywhere: the field at a node is computed from its two neighbors along each axis. This is second-order accurate and is the correct thing to do anywhere in the vacuum between electrodes.

Electrode-aware differs only at metal surfaces. At a node that lies on a conductor and has vacuum on exactly one side along an axis, the central difference straddles the electrode — and because the

potential inside a conductor is flat, the half of the stencil inside the metal contributes nothing. The result is a surface field roughly half its true value. The electrode-aware method replaces that one derivative with a one-sided difference taken into the vacuum, never across the metal interior. Vacuum nodes are untouched, so the two methods differ only on the one-cell skin where the metal faces the gap.

6.2 When each is appropriate

Situation	Use	Why
Ions born on an electrode surface	electrode-aware	The ion's first acceleration happens in exactly the cells where the plain gradient is wrong. A surface-born ion is mis-accelerated by roughly one percent of the gap energy without the correction — small, but systematic, and it maps directly onto arrival-time error.
Ions grazing or striking surfaces	electrode-aware	Whenever the trajectory samples the surface skin, the surface field is part of the answer.
Transport through a central gap	plain gradient	If ions stay away from the electrodes — the usual case for a guide, a funnel, or a transport stack — the correction touches cells the ions never visit. It is then pure cost.
Large grids where speed matters	plain gradient	The correction is a per-metal-node pass, so its cost scales with the electrode surface area on the grid.

The default is electrode-aware, because being right at surfaces is the safer default and the situations that need it are the ones where being wrong is least visible. Choose the plain gradient deliberately, when you can say why the ions do not sample the surfaces.

A related option controls the **stored precision** of the drive-field arrays. The gradient is always computed in double precision; the option decides whether the result is stored as 32-bit or 64-bit. Single precision halves the memory of the field arrays, which becomes the binding constraint on large three-dimensional grids, at a rounding-level cost that is negligible against the discretization error already present. It is stated as an option rather than chosen for you because a frozen reference result stored at one precision will not reproduce bit-for-bit at the other.

7. Flying ions

7.1 The integrator

Trajectories are integrated with the classical fourth-order Runge-Kutta method, the same scheme commonly used for trajectory integration **[1]**. The field is sampled at the ion's position by trilinear interpolation between the eight surrounding grid nodes, which is what makes the grid pitch the dominant accuracy parameter: the interpolated field is smooth, but it is smooth between samples of a field that was itself computed on that grid.

The time step is declared rather than adapted. This is a current design choice that could be modified in the future, though the present approach is clean and valid. The practical consequence is that the step must be checked against the fastest motion in the problem — for an RF device, against the RF period, not against the transit time. A step that comfortably resolves the drift will silently misrepresent the micromotion.

The reason is worth spelling out. A Runge-Kutta step assumes the force varies smoothly ACROSS the step: its error per step grows steeply with how much the field changes within one Δt. Motion in a 3D RF trap or a tight lens is oscillatory, and sampling an oscillation with too few points per cycle does not merely blur it — it aliases it, producing a trajectory that is systematically wrong (spurious heating or

damping, a shifted secular frequency) while looking perfectly smooth. That is why the recommended rule is ~20 steps per RF period, counted against the FASTEST drive present, and why a vacuum TOF drift tolerates a far coarser step than the same ion inside an RF device.

Convergence in Δt is checked the same way convergence is always checked: halve the step, re-fly, and compare the observable you actually care about (a time-of-flight, a transmitted fraction, a spot size). If the answer moves, the first step was too coarse; if it does not, you have measured — not assumed — that the step is converged. The toolkit will not do this for you, deliberately: which observable must be converged is a physics judgment, not a solver setting. However, there are recommendations returned to the user in the UI that estimate the time step needed.

7.2 Time-varying drives

Electrodes are grouped, and each group carries a waveform: sine, cosine, square with an adjustable duty cycle, or an arbitrary tabulated shape. The field seen by an ion is the static field plus the sum of each group's field weighted by its waveform at that instant. This is a full time-domain treatment — the RF is integrated as it actually varies, not approximated — and it is what makes traveling-wave devices expressible: a traveling wave is a set of groups with a phase progression.

7.3 Collisions with background gas

Two gas models are available, and they occupy opposite ends of the pressure axis. HS treats collisions as discrete events: between collisions the ion flies the electric field unperturbed; at each step a collision probability is computed from the hard-sphere cross-section, the gas density and the relative speed, and when a collision fires, a gas atom is drawn from the thermal distribution and the pair scatter elastically. Every kick is individually resolved, which is exactly right when collisions are sparse — a collision cell at tens of millitorr, a funnel entrance — and increasingly expensive and unnecessary as pressure rises and thousands of collisions occur per RF cycle.

SDS — the Statistical Diffusion Simulation of Appelhans and Dahl [4] — is the other end of the axis. Rather than tracking individual collisions, SDS applies two effects per time step: a viscous damping of the acceleration toward the local mobility-limited drift velocity, and a random-walk displacement drawn from tabulated diffusion statistics scaled by the local mean free path. This is a faithful port of the published model, and its assumptions are inherited with it — in particular it is a statistical treatment valid when many collisions occur per time step, not a single-collision model.

The choice between them is therefore a physics statement, not a preference: HS where individual collisions are the phenomenon (energy relaxation, collision counting, low pressure), SDS where the gas acts as a continuum drag-and-diffusion bath (mobility-dominated transport at roughly a Torr and above). Running SDS at a pressure so low that few collisions occur per step violates its founding assumption, and running HS deep into the mobility regime is paying per-collision cost for a statistically determined answer.

Both models read the same declaration. Pressure is authored in torr (P_torr — the canonical pressure field; the derived pascal field of older files is retired), temperature in kelvin, and the gas is chosen by name from the shared table — He, H2, N2, air, Ar, CO2, Kr, Xe — or characterized explicitly by cross-section (HS) and molecular diameter (SDS). A uniform drift of the gas can be declared as a flow vector (collisions.flow_mm_us, in mm/μs): collisions are then sampled in the moving gas frame on every route, which is how a drift tube's counterflow or a funnel's carrier flow enters the physics. A planar (2-D) model refuses a flow component along its unsolved axis rather than silently projecting it away.

Two measured properties of these models are worth knowing before quoting numbers from them. HS timing observables carry a small O(Δt) bias — roughly 1.5–3% per nanosecond of step on reference devices — so demonstrate a Δt plateau before certifying a drift time. SDS reproduces realistic mobilities, but its diffusion does not close the Einstein relation: D/K measures 2.1–2.3 × kT/q on this implementation, a property of the published model itself (empirically fitted drift paired with geometric displacement tables), so SDS spreading estimates carry that factor by construction. Notebook V06

measures both statements against the shipped kernels and asserts bands around them, so a silent change to either model fails a notebook rather than a reader.

Bookkeeping follows the physics: under HS each ion's collision count is recorded (n_col); under SDS it is deliberately absent — the model has no discrete collision events to count, and recording 0 would read as a vacuum flight. The application likewise derives the two step-size ceilings — resolving the fastest drive, and sampling collisions no more often than about one per step — from the declared gas and drive, and warns when the declared Δt violates either; a deck flying SDS below its validity regime (too few collisions per RF cycle) gets a named advisory quoting the number it computed.

7.4 Units

One unit convention runs through the whole toolkit. Mixing these up is the most common source of results that are wrong by exactly a factor of a thousand:

Quantity	Unit
Length	millimeters (mm)
Time	microseconds (µs)
Velocity	mm/µs (numerically equal to km/s)
Potential	volts (V)
Electric field	volts per millimeter (V/mm)
Energy	electronvolts (eV)
Mass	daltons (Da); the ion mass list carries masses, and charge is a separate signed integer
Pressure / temperature	Torr / kelvin

The physical constants behind all of this live in one module (ion_gym.physics.constants — CODATA 2018 and exact SI-2019 values): every kernel, builder and notebook reads the same e, kB, NA and atomic mass constant, so a quantity computed two ways cannot disagree in the fifth digit because two files remembered different vintages.

7.5 Stations: detectors and apertures

A flight that nothing measures is a demonstration, not an experiment. A station is a declared plane in the instrument — carried by the specification like everything else (the top-level stations list) — at a position on one axis, with optional rectangular windows over the other two. There are two kinds, named for what they are physically:

- detect — a measuring plane. Its on_hit field says what a hit does: "pass" logs the crossing and lets the ion continue (the arrival time is the first in-window crossing, interpolated exactly between the recorded points that bracket it); "splat" is an absorbing detector face — the ion ends there, fate 6 (station detect).
- impact_plane — a physical plate. An ion crossing inside the window passes through the aperture; outside it, the ion splats on the plate, fate 5 (station impact plane). An empty window is a solid wall.

Mind the inversion between the two kinds, because it is the one honest trap in this vocabulary: a detect window is where ions are absorbed (a detector patch sitting on the beam), while an impact_plane window is where they are not (a hole in a plate). The same numbers mean opposite fates under the two kinds, and clearing the window turns a plate into a wall — not into a full-plane detector. The station editor's reference text states this beside the controls.

Stations are honored in the kernel on every route — planar, r-z, full 3-D, and every stage of a multi-stage instrument, where a stage's stations are authored in that stage's own frame, exactly as when the stage flies alone. Termination is at the exact interpolated crossing, not at the next recorded sample. Fates 5 and 6 take part in the same fate vocabulary, colors and statistics as every other outcome, and

the Impact Analysis tab reads arrivals at a detect station the same way for a single stage and for an assembly.

7.6 Beyond the solved domain: the ballistic extension

Field-free flight is a straight line, and a straight line's crossing of a plane is exact algebra. So when a flight leaves the solved domain through a face with a declared bound or station beyond it — the detector far downstream of a lens, say — the trajectory is extended by one analytically exact record to honor the declaration: a reachable terminating bound ends the flight exactly on its plane; a wall or detector patch beyond the domain stops the ion with its own fate; a pass detector simply gets its crossing. No field is invented to do this, and the extension refuses by name when its premise fails: when background gas is declared (field-free is not collision-free), or when the escape face still carries residual field above a stated ceiling (the fringe has not decayed there — pad the domain instead).

A deck that declares nothing beyond its domain is untouched by any of this: leaving the box is then simply the fate the flight reports.

8. The pseudopotential view

An RF field exerts a net focusing force on an ion because the field is inhomogeneous: over one RF cycle the ion spends slightly longer where the field is weaker. Averaged over the cycle this is equivalent to a static effective potential — the pseudopotential of Dehmelt **[5]** — of magnitude

$$V^* = q E_0^2 / (4 m \Omega^2)$$

where E_0 is the amplitude of the RF field at that point, m the ion mass, and Ω the angular drive frequency. The toolkit displays the sum of the DC potential energy and this term as an effective potential-energy surface, which is the natural way to see where ions are confined and how deep the well is.

This picture is an approximation and the flight is not. Trajectories are always integrated in the real time-varying field; the pseudopotential is a visualization. It is valid in the adiabatic limit, where the field changes little over one RF period as seen by the ion — the condition quantified by the adiabaticity parameter of Gerlich **[6]**. It degrades where the field is strongly inhomogeneous over a micromotion amplitude, near electrode surfaces, at low *m/z*, and near the edges of stability. When the pseudopotential says an ion is confined and the trajectory says it is lost, the trajectory is the answer.

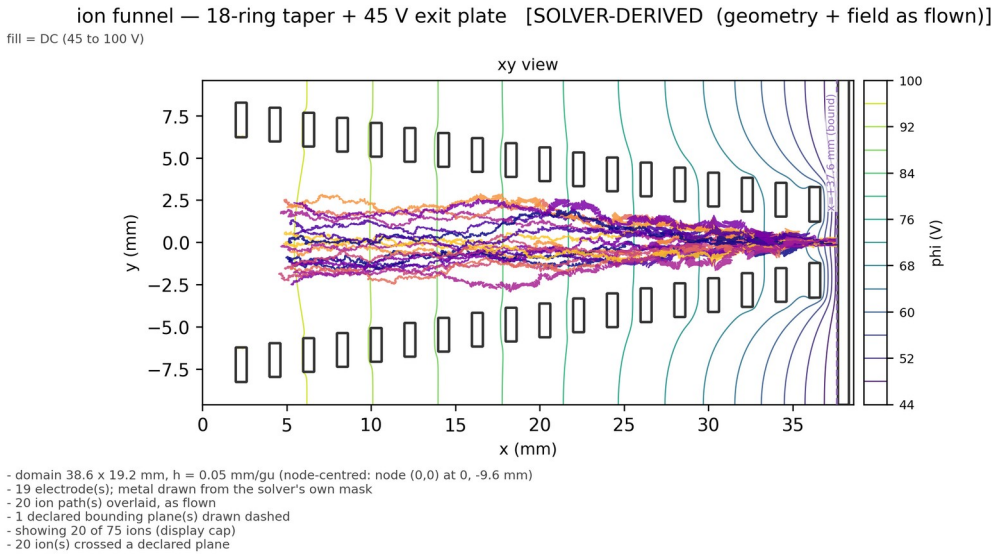


Figure 3. An electrodynamic ion funnel [7]: a stack of ring electrodes of decreasing aperture, driven with RF on alternating rings and a DC gradient along the axis. The RF confines ions radially through the pseudopotential while the DC pushes them forward; the geometry is the solver's own electrode mask, not a schematic.

9. Reading results

A flight returns a trajectory array and a summary. The trajectory array has one row per recorded step; the column names come back from the build, so read them rather than assuming an order. Recorded per step are time, position, velocity, and the electric field components the ion actually experienced at that point. From these the interface derives further channels — speed, kinetic energy in eV, radial position, field magnitude — which can be plotted against one another for a single ion or a whole population.

The summary reports how the flight ended, and that outcome deserves attention: an ion that timed out is not the same as one that struck an electrode or left through a declared boundary, and a population statistic computed without separating those is usually meaningless.

The fate vocabulary is short and worth memorizing — every view, legend and statistic uses the same codes:

- 0 impact / exit — the ion struck metal (or left through a declared aperture in it);
- 1 boundary exit — it left the solved domain;
- 2 timeout — still in flight when the declared maximum flight time expired: not a loss, a question about your clock;
- 3 bounding plane — it crossed a declared terminating bound;
- 4 transporter max_passes — it reached a declared pass-count ceiling in a looping transporter;
- 5 station impact plane / 6 station detect — it ended on a declared station (section 7.5).

For a multi-stage instrument one more rule applies, and it is a definition rather than a computation: the only transmission figure is how many of the ions flown hit the detector. Everything else — hit metal, timed out, left the box — is a named reason an ion was not detected, and an instrument that declares no detector has flown ions but measured nothing (section 10).

Recording is subsampled by a declared interval. This decision is adjustable by the user and an option to save memory. Additionally, every recorded quantity carries its unit in the interface, and the units are those of section 7.4 — in particular the field channels are in $\mathrm{V/mm}$, not V/m.

10. Multi-stage instruments (assemblies)

A real instrument is rarely one field. A funnel at a few torr hands ions to a multipole at millitorr; an orthogonal accelerator pushes a packet into a mirror. Each region wants its own grid, its own pitch, its own gas and its own clock — forcing them into one solve would mean solving the whole instrument at the finest pitch and the highest pressure found anywhere in it. An assembly instead keeps each stage its own solved model and flies ions across declared seams.

The assembly document is JSON like everything else: a list of stages — each carrying a complete specification of its own, a pose placing it in the world (offset and rotation), and an exit plane where it hands the ion to the next stage — plus a beam block saying what is flown. The shipped example is examples/funnel_hexapole/instrument.json: an RF funnel posed into a tilted-wire hexapole, with a detector at the hexapole's exit. Notebook 11 works it end to end through the API.

Seams are checked, loudly, before anything flies. A handoff is supposed to be field-dead: the preflight measures the worst residual field an ion can see at each seam and states the number as an advisory

— quantified, never blocking. A declared bound that sits inside its own stage's exit plane is refused outright: every ion would terminate on the bound before reaching the seam, so the handoff is dead by construction.

Each stage flies on its own terms. Its clock is its own declared maximum flight time — a timeout names the stage whose clock expired, and raising another stage's does nothing. Its recording density and buffer are its own. Its gas is its own, so a gas-filled funnel feeds a vacuum analyzer without either compromising. Its stations and bounds are authored in its own frame, exactly as when the stage is flown alone. What crosses the seam is the ion's state: position, velocity, time.

The beam is generated, not pasted. The usual declaration is from_stage: the packet is produced from the named stage's own source block at load, so the instrument flies the same births that stage would fly alone. The ion-count field in the application then narrows that packet for a quick look — an even stride across the declared distribution, never just its head — or, when the packet carries its generator's stamp and seed, regenerates it at a larger count from the same declaration. Duplicating rows is refused: two identical ions fly identical paths and would double-count every histogram.

What comes back is counted by one rule (section 9): detected / flown. The status line reads, for example, "70/75 detected", followed by the named reasons for the rest — hit metal in the funnel, left the hexapole's box, timed out — and an assembly with no declared detector says so instead of printing a number: ions were flown, nothing measured them. Time-of-flight statistics and resolving power are computed from detected ions only, per mass; a spread pooled across masses would dress mass separation up as peak width, so the headline refuses to pool it.

Before the first ion flies, the same record-volume guard that protects a single-stage flight prices what an assembly flight will retain — the drawn trajectory subset — against your declared memory budget, refusing with the arithmetic and the levers when it cannot fit (section 11).

In the application, the Multi FA tab drives all of this: paste or load the instrument JSON, pick which stage the view and the flight parameters address, and Fly assembly launches the solve-and-fly on a worker thread with progress streaming to the status line — the first 3-D solve compiles kernels and can take minutes cold while the application stays live. The View pane draws every stage posed in the world frame with the seams marked, and the 3-D Flight Viewer (section 13.2) shows the flown paths inside the full instrument.

11. When the toolkit refuses: reading the guards

A consistent design choice runs through everything above: where a configuration cannot mean what it says, the toolkit refuses by name rather than flying something plausible. A refusal is not a crash — it is the tool telling you which declaration to change, usually with the arithmetic in hand. The ones you will actually meet:

The trajectory record would not fit. Before anything is built, the record's size is quoted exactly — ions × samples × channels × 8 bytes. An oversized run refuses; a merely large one warns and flies. The ceiling is this machine's memory, or the budget you declare (max record RAM on the Config tab; scripts pass record_budget_gb). The cheapest lever is almost always the recording interval: on a single-stage flight it is a storage decision, not a physics one — time of flight, fates, impact positions and collision counts are identical at any recording density.

The recording cannot cover the flight. With the per-ion record cap too low for the declared flight time, a single-stage run announces truncation — the stored trace stops, the flight continues to its true fate, and the summary reports that fate. A staged flight refuses instead, because its arrival times are read off the recorded trace: a crossing after the buffer filled would be invisible, and the ion falsely reported as never detected.

The solve is priced before it runs. Loading an example or changing resolution quotes the node count and the expected solve time for this machine before you commit, and the smallest-feature field warns when the pitch cannot resolve the gap you care about (less than about three cells across it).

A station's window names its own axis, or its pair is inverted. Both refuse: a plane cannot be windowed along its own normal, and a backwards (lo, hi) pair would silently turn a wall into a pass-through.

A declared bound sits inside its stage's exit seam (assemblies): refused as a dead handoff — section 10.

The ballistic extension's premise fails (section 7.6): background gas is declared, or the escape face carries residual field above the stated ceiling. The message quotes the measured field; the remedy is padding the domain, not raising the ceiling.

An ion is born inside metal. That ion splats at birth with a warning and the rest of the packet flies. A configuration error — an unknown gas, a malformed station, a mis-declared seam — aborts the run instead: configuration mistakes must never be tallied as transmission losses.

The solver did not converge. The iteration cap was reached above tolerance, and the field is not returned. An unconverged field that looks plausible is the most dangerous object in this toolkit, which is why you get an error instead of a result.

The mesh on disk is not the mesh the specification hashed. The STL preflight refuses on a SHA-256 mismatch (section 4.2): a re-exported part is a new geometry and must look like one.

Advisories are the softer form — printed once per run, never blocking: the point-source note, the Δt-bias and SDS-regime notes (section 7.3), the seam residual fields (section 10). An advisory you have read and can argue with is fine to fly past; one you cannot explain is the simulation telling you it is about to answer a different question than you asked.

12. Limitations

The following are properties of the ion_gym approach, not defects. Knowing them is the difference between a simulation you can defend and one you cannot.

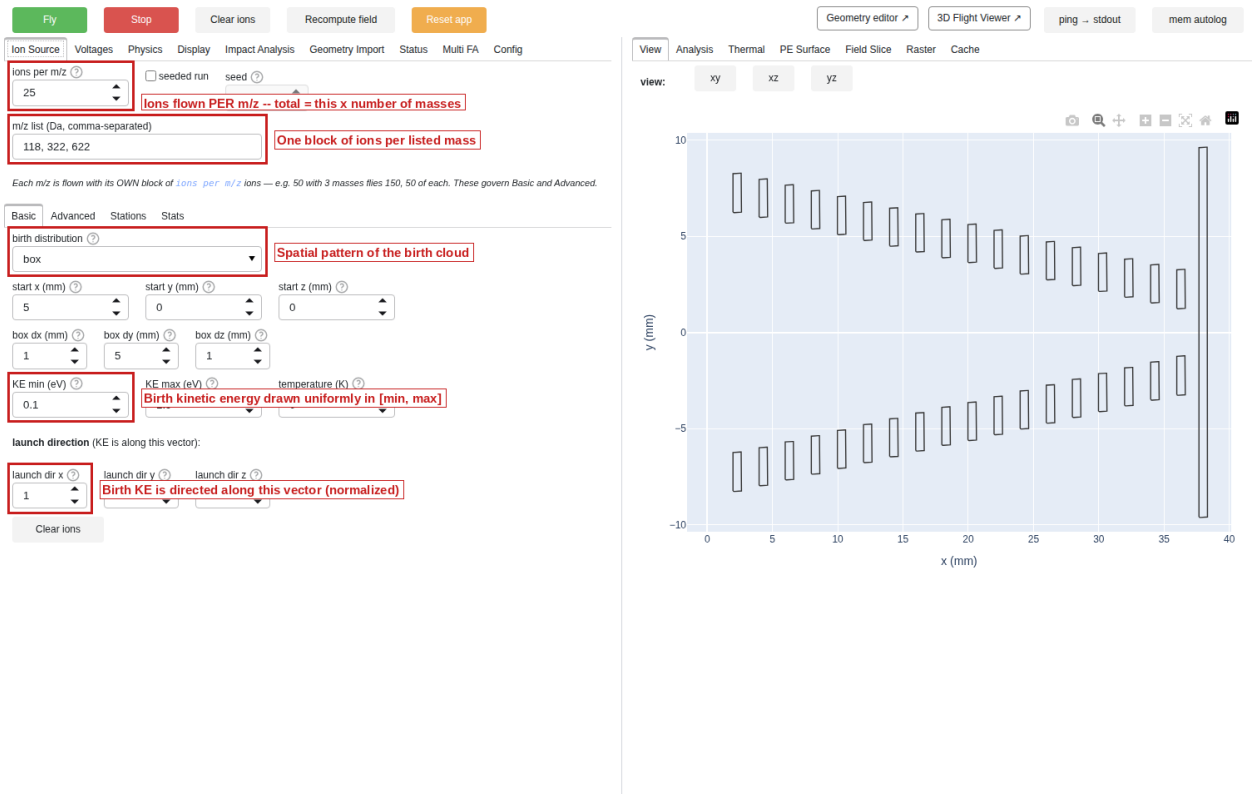
- **Electrostatics only.** Laplace's equation with conductors at fixed potentials. No magnetic fields. No dielectric materials. No space charge, so the results are single-ion results: they do not describe a beam dense enough for ion-ion repulsion to matter, and they will not reproduce space-charge limits on transmission or trapping capacity.
- **A uniform grid.** Everything is discretized at one pitch. Accuracy is governed by that pitch, and geometric detail below roughly two cells is not resolved. There is no local refinement, so a fine feature inside a large domain forces a fine pitch everywhere and the cost grows as the cube of the refinement in three dimensions.
- **Voxelized surfaces.** A curved or angled electrode becomes a staircase of grid cells. This matters most where the surface field matters most — near sharp features and in narrow gaps — and it is a discretization error that no choice of gradient method removes.
- **A fixed time step.** The integrator does not adapt. The error of a Runge-Kutta step grows steeply with how much the force changes within the step, so the step must be chosen against the fastest motion present — the RF period in a driven device, the sharpest field gradient in a static one — not against the transit time. An adaptive integrator would detect a too-coarse step and shrink it; this one will integrate an undersampled oscillation into a smooth, plausible, wrong trajectory. Verifying convergence is therefore the user's responsibility, and the check is mechanical: halve Δt, re-fly, and confirm the observable you are reporting does not move (section 7.1).
- **Statistical collisions. The gas model is a diffusion-and-mobility treatment valid in the many-collisions-per-step regime. A single uniform drift of the gas can be declared, but there is no computed flow field: jets, expansions, and pressure gradients across one region are out of scope, and a step change in pressure between the stages of an assembly is an idealization of a differential-pumping aperture, not a simulation of one.**

- **The pseudopotential is a picture.** As described in section 8 it is an adiabatic approximation used for display. Its RF term is built from the field gradient within the displayed plane, so on a slice through a structure that varies steeply out of that plane it understates the true confinement.
- **Symmetry is a declaration.** A mirror plane is applied because the specification asserts it. Asserting a symmetry the voltages do not actually have produces a field that is wrong in a way the solver cannot detect.

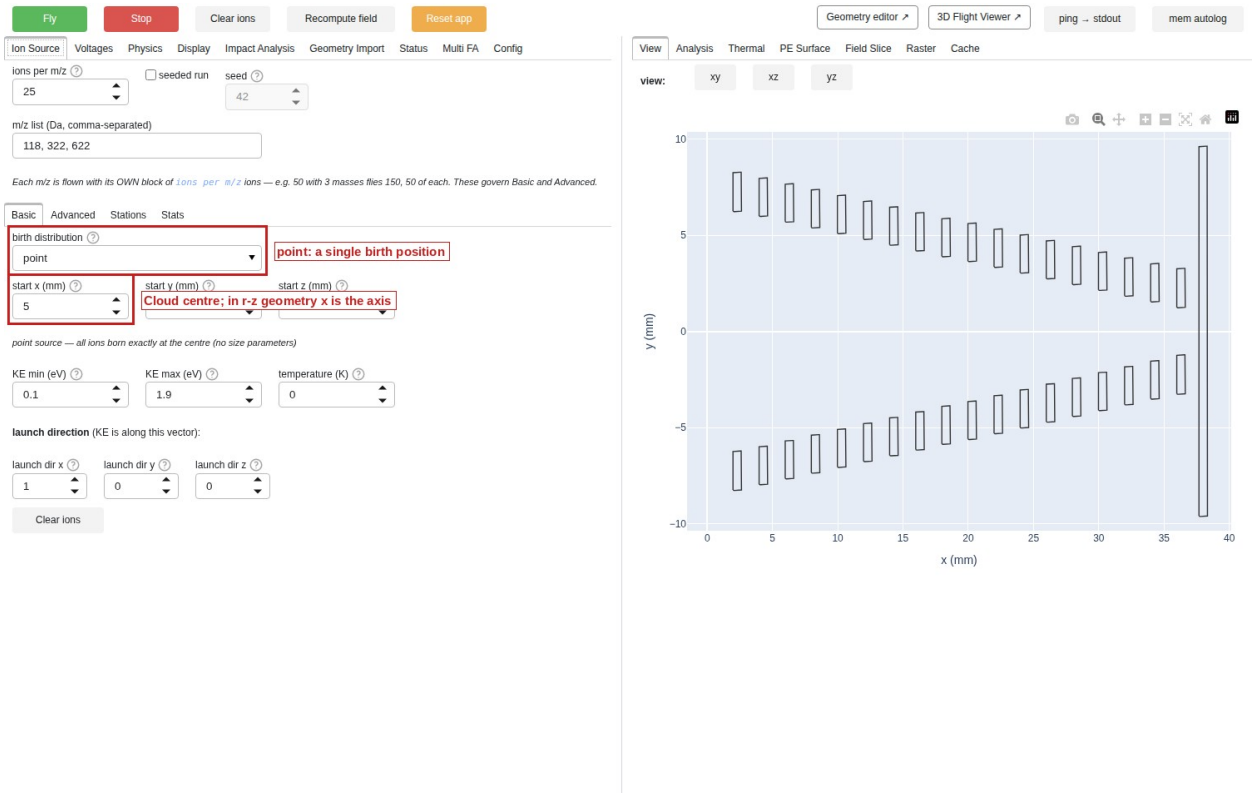
13. Dashboard reference (tab by tab)

Every control below is documented from the application itself: the UI harvester walks the live widget tree and emits this section from the widgets' own description strings, so this section cannot drift from the app. Hover any control in the dashboard for the same text.

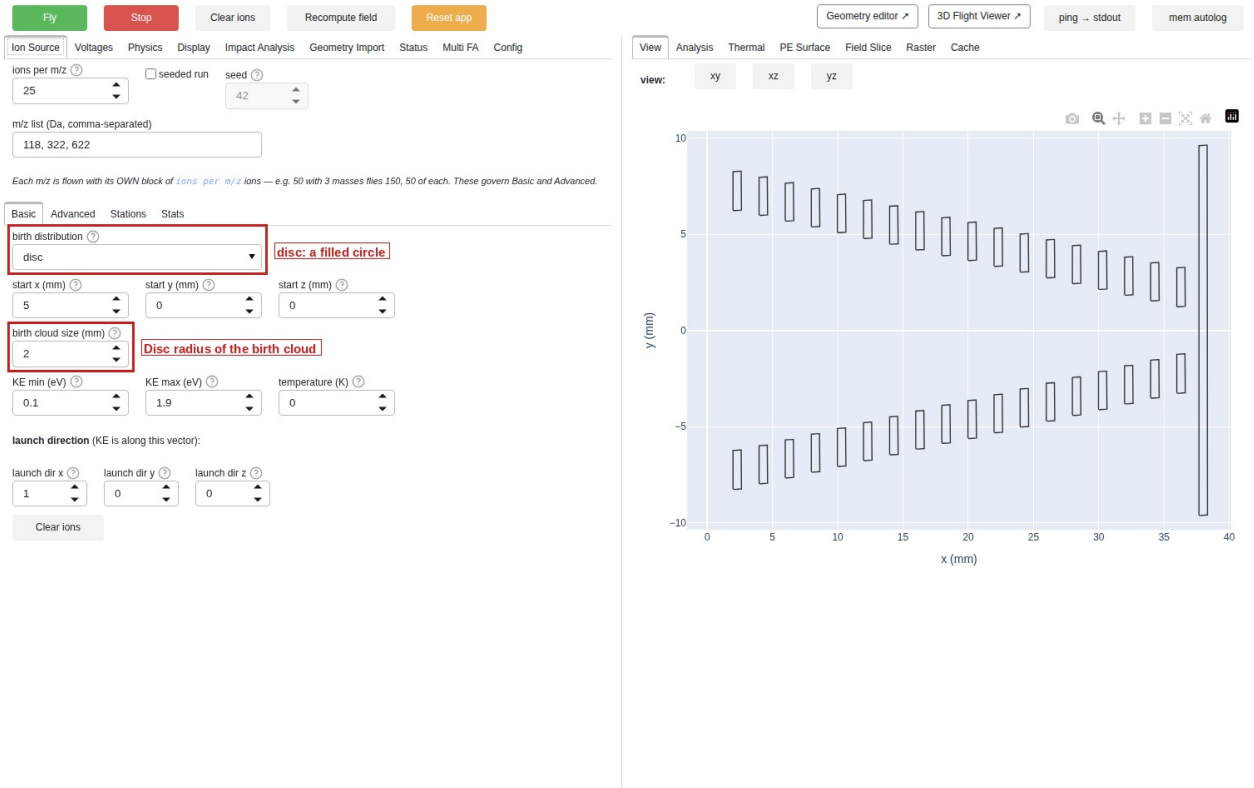
Ion Source



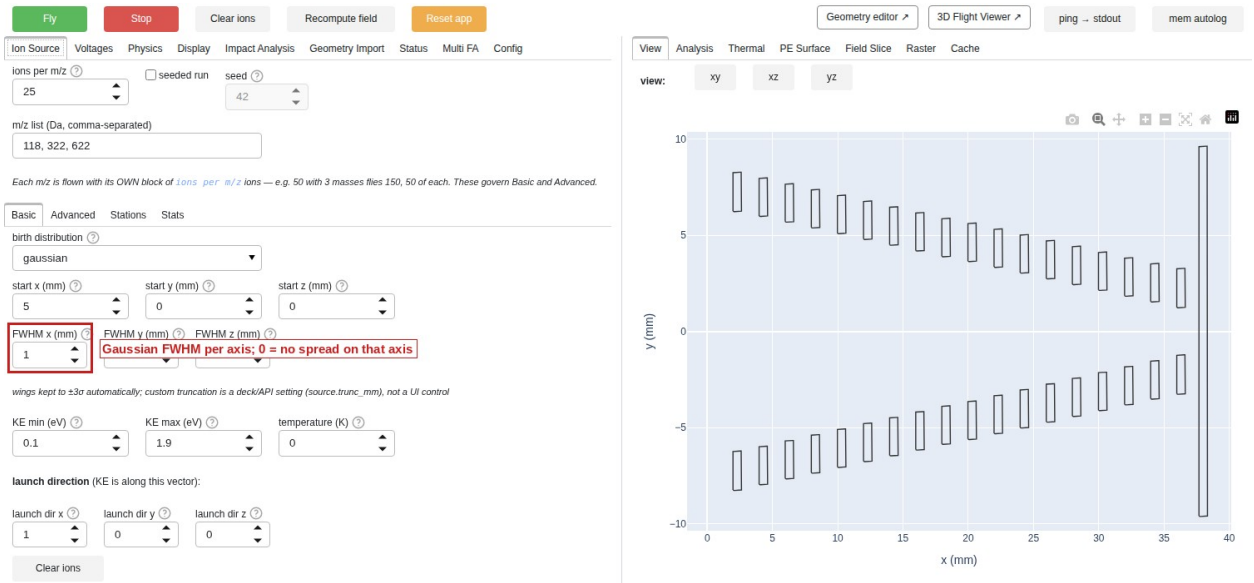
Ion Source tab, box birth distribution (the default): ions are placed uniformly inside a rectangular volume whose extents are the box dx/dy/dz fields.



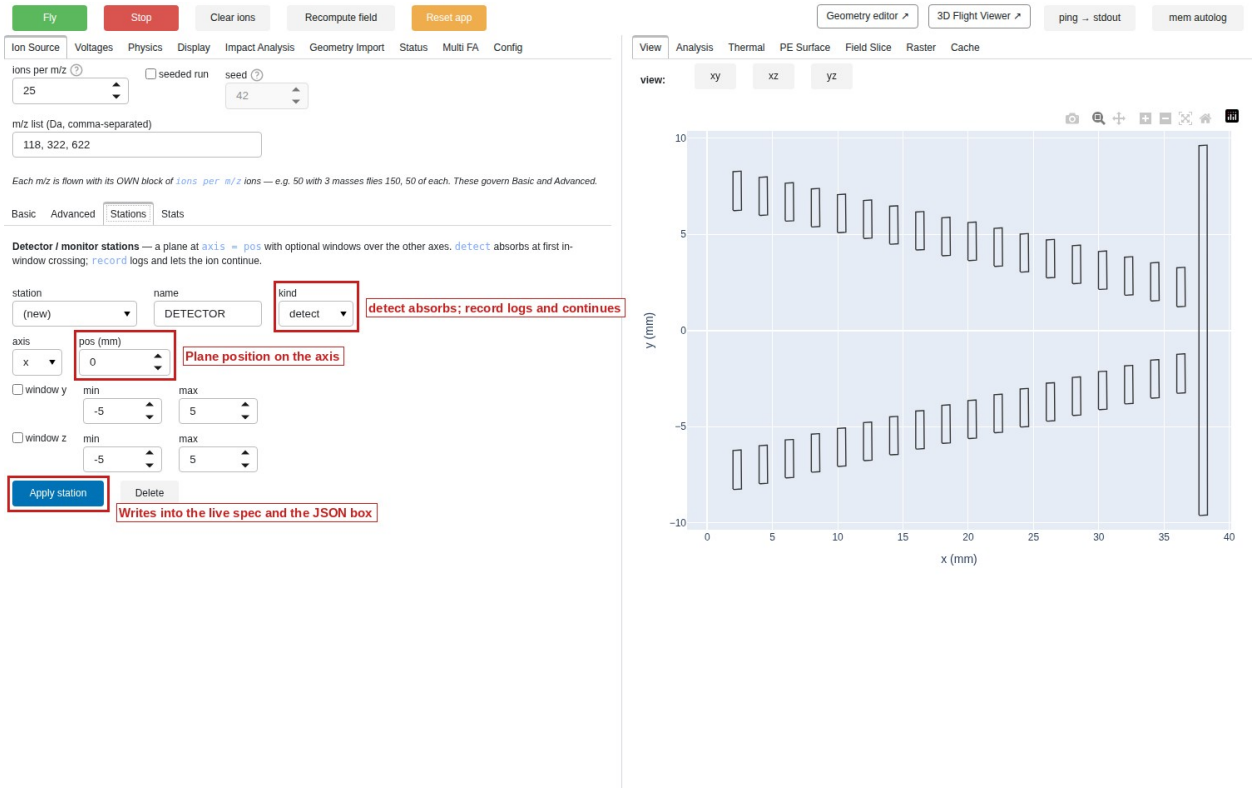
Ion Source tab, point birth distribution: zero declared extent, so downstream spread is set entirely by the optics (the app prints a point-source advisory for exactly this case).



Ion Source tab, disc birth distribution: a filled disc of the given radius around the start position.



Ion Source tab, gaussian birth distribution: per-axis FWHM; truncation is derived at 3 sigma -- an untruncated Gaussian is a different beam, and the derived cut removes the wings without another knob.



The station editor (Ion Source > Stations): a detector or aperture plane at axis = pos with optional windows over the other two axes. Kind detect carries on hit: pass logs the first in-window crossing and the ion continues; splat absorbs it there (fate 6). Kind impact_plane is a plate: in-window passes through the aperture, outside splats (fate 5), and an empty window is a wall. Apply station writes into the live specification and the JSON box. (The capture above predates the on-hit control; section 7.5 has the current vocabulary.)

- **ions per *m/z* (IntInput) —** Ions flown PER *m/z* value. Total flown = this x the number of *m/z* entries. E.g. 50 with 4 masses flies 200 ions (50 of each).
- seeded run (Checkbox)
- **seed (IntInput) —** Pins the whole run: births, birth times and collision draws. Ion i always draws the same numbers, so reruns are bit-identical and two tunes are compared on common random numbers. Unchecked (default): fresh entropy per run, drawn seed printed.
- *ion mass list (Da, comma-separated) (TextInput) — Masses in daltons, not mass-to-charge: charge is the separate field below.*
- charge state (signed z) (IntInput) — Signed integer charge state for every ion in the packet. 0 refuses at fly time.
- on hit (detect) (Select) — detect only: pass = log and continue; splat = absorbing detector (fate 6). An impact_plane ignores this (aperture passes, plate splats, by definition).
- **birth distribution (Select) —** Spatial pattern the ions are seeded over: a single point, a filled disc, a line, a grid, or a BOX (random uniform placement inside a rectangular volume -- avoids the skew of a contrived single-point source; extents set below).
- **start x (mm) (FloatInput) —** Birth position of the ion cloud center, x (mm). In r-z geometry x is the axis.
- **start y (mm) (FloatInput) —** Birth position of the ion cloud center, y (mm). In r-z geometry y is the radius.
- **start z (mm) (FloatInput) —** Birth position z (mm) — relevant for 3-D scenes.
- **box dx (mm) (FloatInput) —** Full extent (mm) of the birth box along this axis, centered on (start x, y, z); each ion is placed uniformly at random inside. 0 collapses that axis.
- **box dy (mm) (FloatInput) —** Full extent (mm) of the birth box along this axis, centered on (start x, y, z); each ion is placed uniformly at random inside. 0 collapses that axis.
- **box dz (mm) (FloatInput) —** Full extent (mm) of the birth box along this axis, centered on (start x, y, z); each ion is placed uniformly at random inside. 0 collapses that axis.
- **KE min (eV) (FloatInput) —** Lower bound of the birth kinetic energy; each ion draws uniformly in [min, max]. Set equal for monoenergetic.
- **KE max (eV) (FloatInput) —** Upper bound of the birth kinetic energy.
- **temperature (K) (FloatInput) — Isotropic Maxwell-Boltzmann thermal spread of the birth velocities, superposed on the directed beam KE (a beam AT a temperature). 0 = off. This is the turn-around-time knob in a TOF source.**

launch direction (KE is along this vector):**

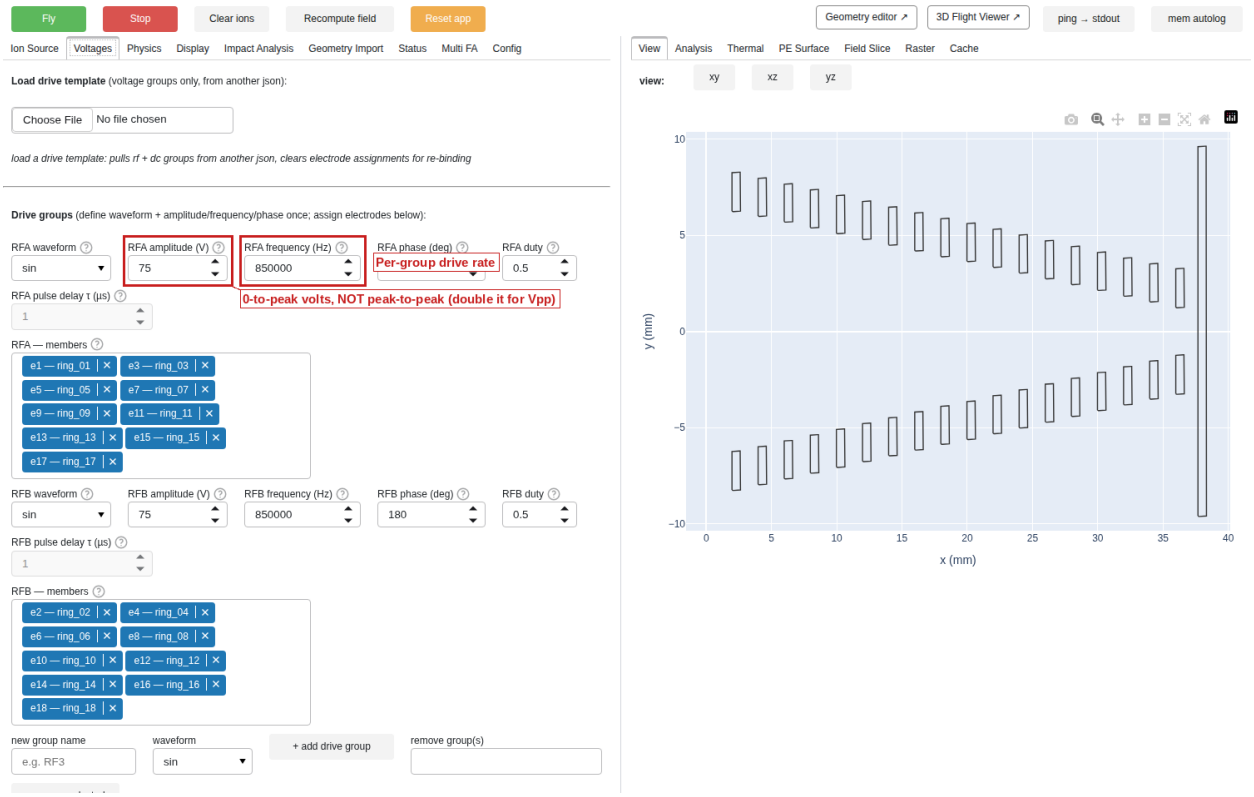
- **launch dir x (FloatInput) —** Launch direction vector (the birth KE is directed along this; it is normalized, so only the direction matters, not the magnitude). e.g. (0,0,1) fires along z.
- **launch dir y (FloatInput) —** Launch direction vector (the birth KE is directed along this; it is normalized, so only the direction matters, not the magnitude). e.g. (0,0,1) fires along z.
- **launch dir z (FloatInput) —** Launch direction vector (the birth KE is directed along this; it is normalized, so only the direction matters, not the magnitude). e.g. (0,0,1) fires along z.
- **Clear ions (Button) —** Clear the flown ions from the view (same as the top-bar Clear ions).

Beam declaration

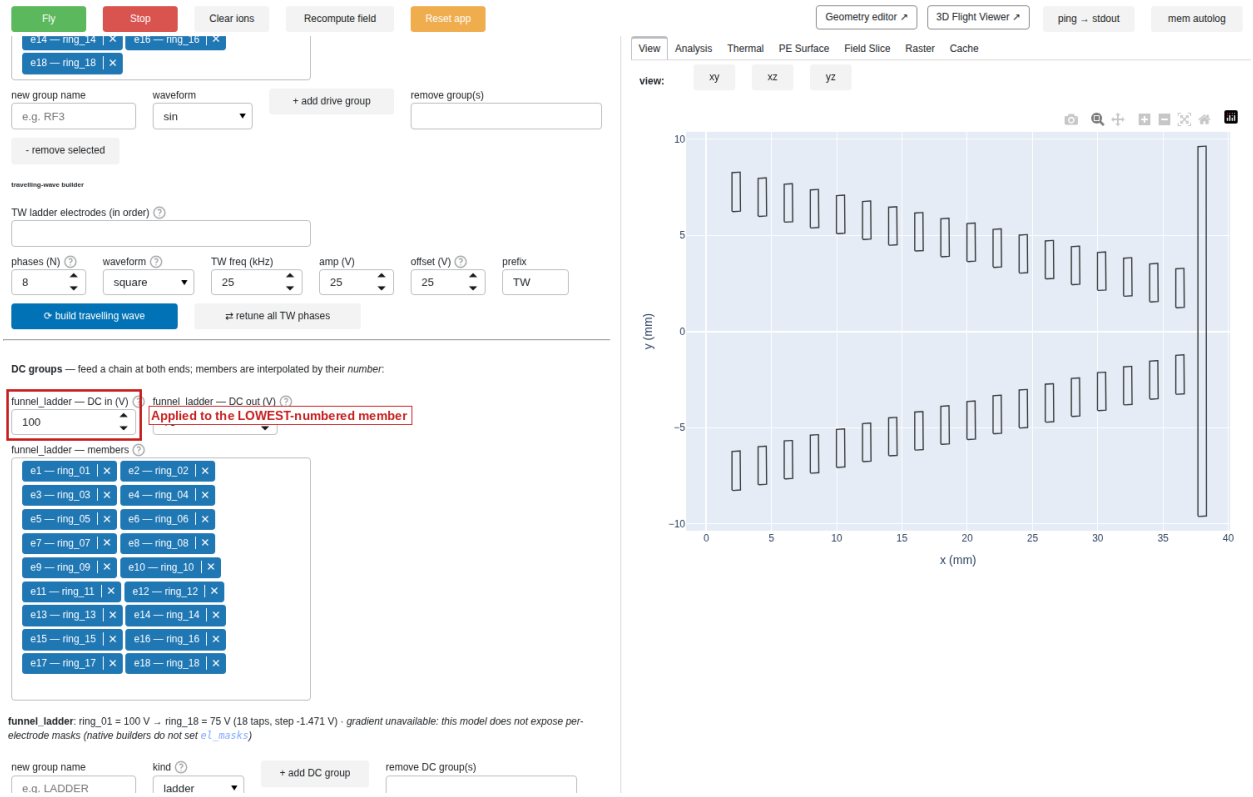
Below the birth-cloud controls sits the beam declaration: a compact statement of the packet as a beam. It fixes the ion count and seed, names the propagation axis, and then gives one row per phase-space channel — the two transverse positions, the three direction components, the kinetic energy and the birth time. Every row has the same shape: a distribution chooser and its one or two parameters, so declaring a beam reads as a table of channels rather than a scatter of boxes. Regenerate beam draws a fresh packet from the declaration; Derive from current packet runs the other direction, fitting the declaration to whatever packet is currently loaded — the usual way to turn a flown arrival distribution into the birth condition of the next stage.

The station editor follows (Ion Source > Stations), with the section-7.5 vocabulary: kind detect with on hit pass or splat, kind impact_plane for plates, windows over the two transverse axes. Apply station writes into the live specification and the JSON box, so a save persists it. Last, the Stats sub-tab selectors choose which population the statistics table describes: which surface, which transmitted fate, and which plane the impact cross-section is taken in.

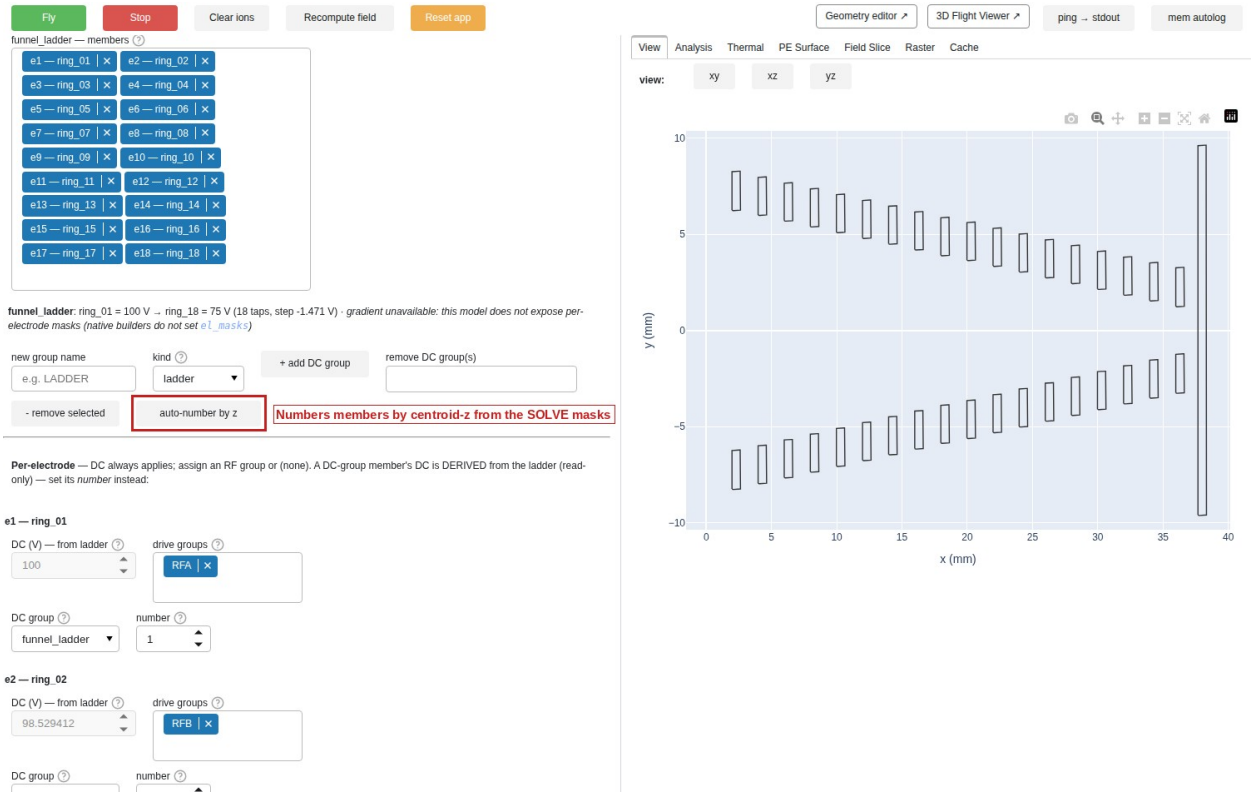
Voltages



Voltages tab, drive groups: waveform, amplitude, frequency, phase and members are defined once per group; electrodes are assigned below.



Voltages tab, DC ladder groups: feed a chain at both ends and members are interpolated by their explicit number -- order is never parsed from names.



Voltages tab, ladder numbering: auto-number by z orders members by centroid-z straight from the SOLVE masks -- it needs a solved field and refuses rather than guessing.

Load drive template** (voltage groups only, from another json):

Drive groups (define waveform + amplitude/frequency/phase once; assign electrodes below):**

- **RFA waveform (Select) —** Drive waveform for this group. 'table' is the general sampled shape (arbitrary); analytic shapes are sin/square (triangle/sawtooth extensible).
- **RFA amplitude (V) (FloatInput) —** Drive amplitude, 0-to-peak volts (a value of 100 here means 200 V peak-to-peak).
- **RFA frequency (Hz) (FloatInput) —** Drive frequency for this group, in Hz. For a stepped traveling wave this is the per-group rate.
- **RFA phase (deg) (FloatInput) —** Phase offset of this group's waveform. A traveling wave is groups stepped in phase; two confinement rails run 180 deg apart.
- **RFA duty (FloatInput) —** Square waves only: fraction of the period HIGH (0.5 = 11110000 across 8 stepped phases; 0.25 = 11000000).
- **RFA pulse delay τ (µs) (FloatInput) —** Step-table drives only: the waveform holds 0 until τ, then 1 (× amplitude). This is Wiley-McLaren delayed extraction when it drives the pusher/extraction plates. NOTE: τ is co-tuned with the mirror detune — changing τ alone moves the velocity focus off the detector. Non-step tables are read-only here.
- **RFA — members (MultiChoice) —** Electrodes driven by this group. Same as the per-electrode 'drive groups' box; an electrode may be in several drive groups and a DC group.
- **RFB waveform (Select) —** Drive waveform for this group. 'table' is the general sampled shape (arbitrary); analytic shapes are sin/square (triangle/sawtooth extensible).
- **RFB amplitude (V) (FloatInput) —** Drive amplitude, 0-to-peak volts (a value of 100 here means 200 V peak-to-peak).
- **RFB frequency (Hz) (FloatInput) —** Drive frequency for this group, in Hz. For a stepped traveling wave this is the per-group rate.
- **RFB phase (deg) (FloatInput) —** Phase offset of this group's waveform. A traveling wave is groups stepped in phase; two confinement rails run 180 deg apart.
- **RFB duty (FloatInput) —** Square waves only: fraction of the period HIGH (0.5 = 11110000 across 8 stepped phases; 0.25 = 11000000).
- **RFB pulse delay τ (µs) (FloatInput) —** Step-table drives only: the waveform holds 0 until τ, then 1 (× amplitude). This is Wiley-McLaren delayed extraction when it drives the pusher/extraction plates. NOTE: τ is co-tuned with the mirror detune — changing τ alone moves the velocity focus off the detector. Non-step tables are read-only here.
- **RFB — members (MultiChoice) —** Electrodes driven by this group. Same as the per-electrode 'drive groups' box; an electrode may be in several drive groups and a DC group.
- new group name (TextInput)
- waveform (Select)
- + add drive group (Button)
- remove group(s) (MultiChoice)
- - remove selected (Button)

traveling-wave builder

- **TW ladder electrodes (in order) (MultiChoice) —** Ordered electrodes the wave travels along. Order is the wave direction; phase steps one electrode per segment. Assignment is cyclic (electrode j -> phase j mod N), so any number of segments needs only N groups.
- **phases (N) (IntInput) —** Number of distinct phases (4 and 8 are common SLIM).
- **waveform (Select) —** square = classic stepped SLIM TW; sin = sinusoidal TW.
- TW freq (kHz) (FloatInput)
- amp (V) (FloatInput)

- **offset (V) (FloatInput) —** DC offset of the drive. Unipolar 0..V SLIM drive = amp V/2, offset V/2 (e.g. amp 25, offset 25 -> 0..50 V).
- prefix (TextInput)
- ⟳ build traveling wave (Button)
- **⇄ retune all TW phases (Button) —** Apply the amp / freq / waveform / offset above to EVERY phase group of the existing ladder at once, keeping each group's phase. Use after 'build traveling wave' to change the drive with one edit instead of N.

DC groups — feed a chain at both ends; members are interpolated by their *number*:**

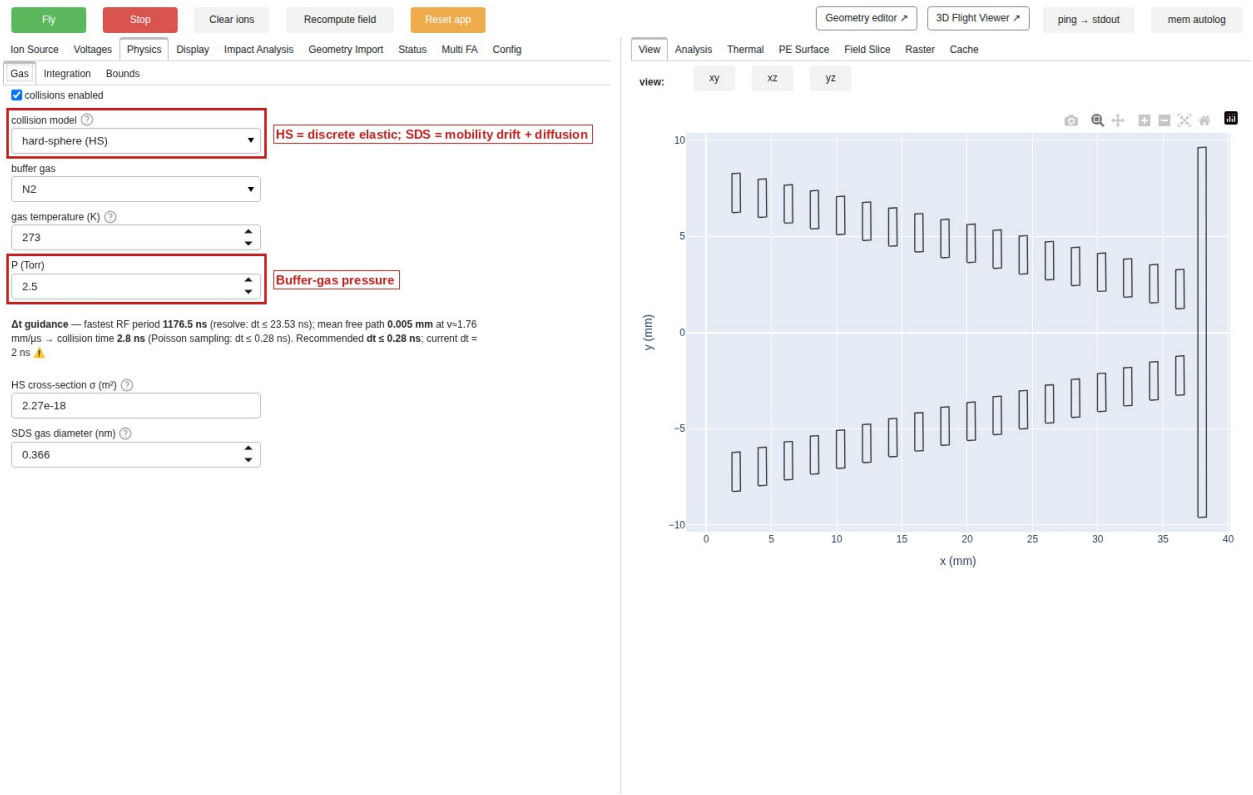
- **funnel_ladder — DC in (V) (FloatInput) —** Applied to the member with the LOWEST number.
- **funnel_ladder — DC out (V) (FloatInput) —** Applied to the member with the HIGHEST number. A RE-WEIGHT, not a re-solve: cache hit.
- **funnel_ladder — members (MultiChoice) —** Electrodes in this DC group. Editing here is the same as the per-electrode 'DC group' dropdown — an electrode may also be in a drive group at the same time.
- new group name (TextInput)
- **kind (Select) —** ladder = two-end interpolated divider (set DC in / DC out and a number per member). uniform = one voltage on every member (no numbering needed).
- + add DC group (Button)
- remove DC group(s) (MultiChoice)
- - remove selected (Button)
- **auto-number by z (Button) —** Number the members of each group in ascending centroid-z, straight from the SOLVE masks. Needs a solved field; refuses rather than guessing.

e1 — ring_01 (and identically for every electrode)

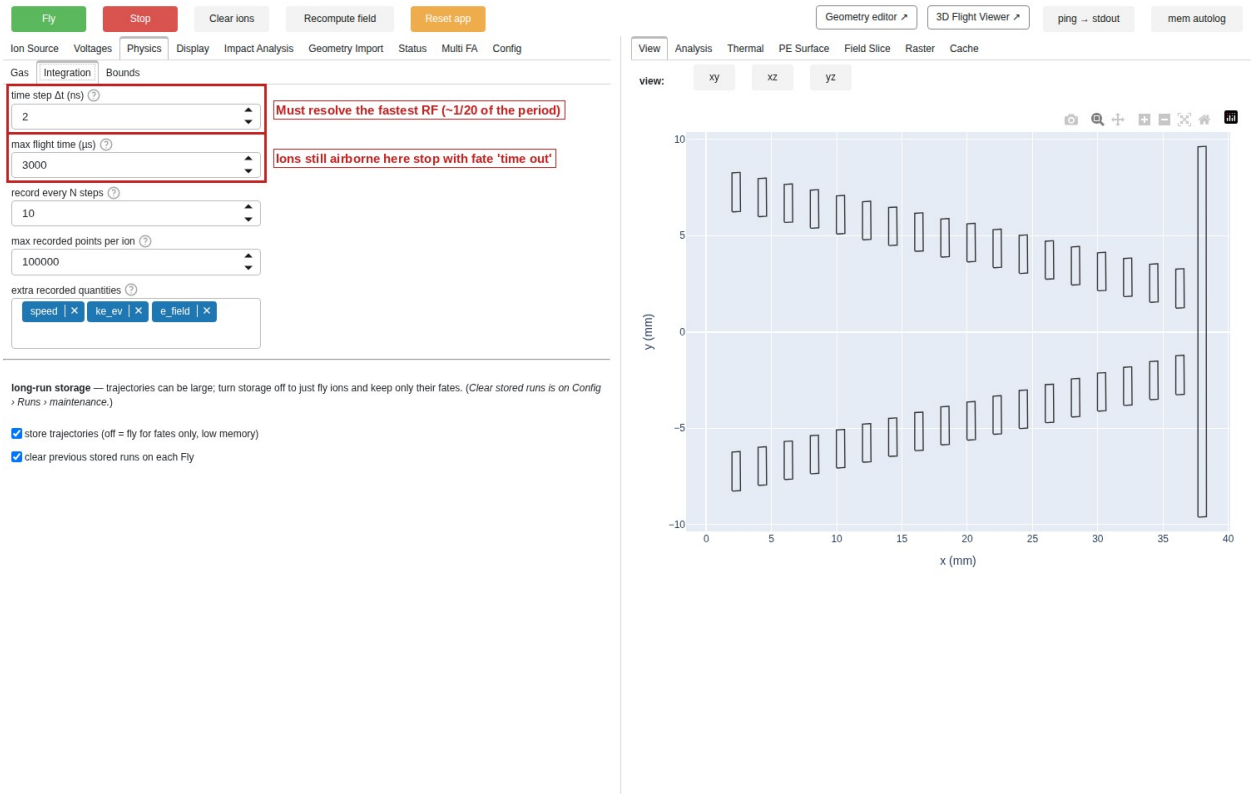
- **DC (V) — from ladder (FloatInput) —** Static (DC) potential on this electrode. When the electrode belongs to a DC group this is DERIVED from the group's in/out and its number.
- **drive groups (MultiChoice) —** Assign this electrode to zero or MORE drive groups (multi-select). An electrode can carry a confinement RF and a traveling-wave AC at once, plus its DC.
- **DC group (Select) —** Ladder membership. DC and RF groups are independent — an electrode can be in both.
- **number (IntInput) —** Position in the ladder. EXPLICIT: order is never parsed from the name (E10 would sort before E9). DC in lands on the lowest, DC out on the highest.

The per-electrode block repeats identically for each electrode in the stack: its DC value (derived from the ladder when the electrode is a member), its drive-group memberships, its DC group and its explicit ladder number. Only the first is shown above; restating nineteen copies would document the layout, not the controls.

Physics



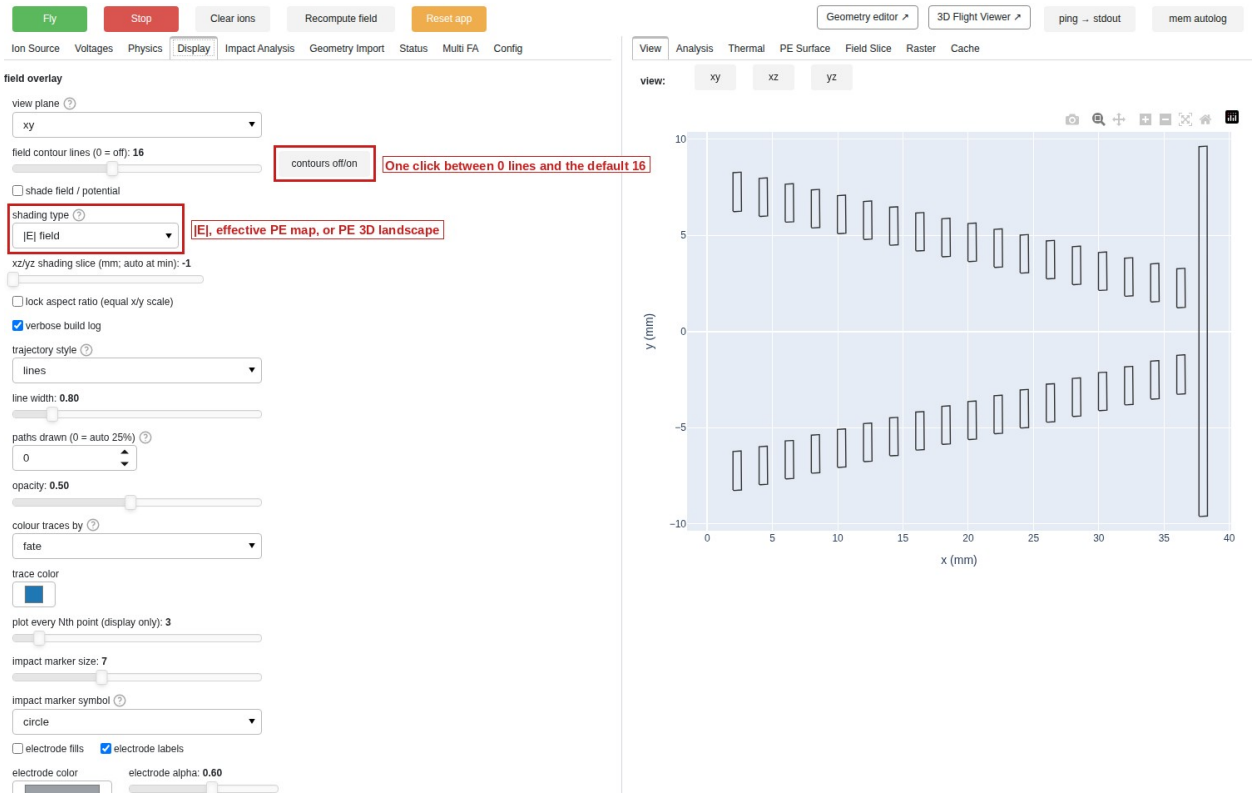
Physics tab, Gas sub-tab: the collision model (HS hard-sphere or SDS diffusion), the buffer gas, its pressure and temperature. The dt guidance line underneath is live -- it derives the resolving and collision-sampling limits from the current gas and drive, and warns when the declared step is too coarse for either.



Physics tab, Integration sub-tab: the integration time step and the flight-time ceiling. This is a separate capture because it is a separate sub-tab -- a callout can only box a control the reader can actually see.

- collisions enabled (Checkbox)
- **collision model (Select) —** HS = discrete elastic hard-sphere collisions. SDS = the Statistical Diffusion model (mobility drift + diffusion), matched to tabulated ion mobilities.
- buffer gas (Select)
- **gas temperature (K) (FloatInput) —** Buffer-gas temperature; sets the thermal velocity ions relax toward.
- P (Torr) (FloatInput)
- **HS cross-section σ (m²) (LiteralInput) —** Hard-sphere ion–neutral collision cross-section. Used only by the HS model; larger σ = more frequent collisions. Typical N_2 value ≈ 2.3e-18 m².
- **SDS gas diameter (nm) (FloatInput) —** Neutral gas molecule diameter, used only by the SDS (diffusion) model to set the mean free path. Air ≈ 0.366 nm.
- **time step Δt (ns) (FloatInput) —** Integration time step. Smaller = more accurate but slower; it must resolve the fastest RF (rule of thumb ≈ 1/20 of the RF period, so ≈0.06 µs at 800 kHz).
- **max flight time (µs) (FloatInput) —** Longest an ion is flown. An ion still airborne at this time stops with fate 'time out'.
- **record every N steps (IntInput) —** Trajectory sampling interval: one point is stored every N integration steps. Larger = fewer stored points (smaller files, faster plots); it does NOT change the physics, only how densely the path is saved.
- **max recorded points per ion (IntInput) —** Trajectory buffer cap. When a flight needs more points than this the RECORDING stops but the ion keeps flying — the app warns before the fly and names the true stopping point afterwards. Raise this (or 'record every N') to capture a whole long flight.
- **extra recorded quantities (MultiChoice) —** Additional per-point values to store along each trajectory (e.g. field magnitude, collision count) for coloring or export.
- store trajectories (off = fly for fates only, low memory) (Checkbox)
- clear previous stored runs on each Fly (Checkbox)
- x min on (Checkbox)
- x min (mm) (FloatInput)
- x max on (Checkbox)
- x max (mm) (FloatInput)
- y min on (Checkbox)
- y min (mm) (FloatInput)
- y max on (Checkbox)
- y max (mm) (FloatInput)
- z min on (Checkbox)
- z min (mm) (FloatInput)
- z max on (Checkbox)
- z max (mm) (FloatInput)

Display

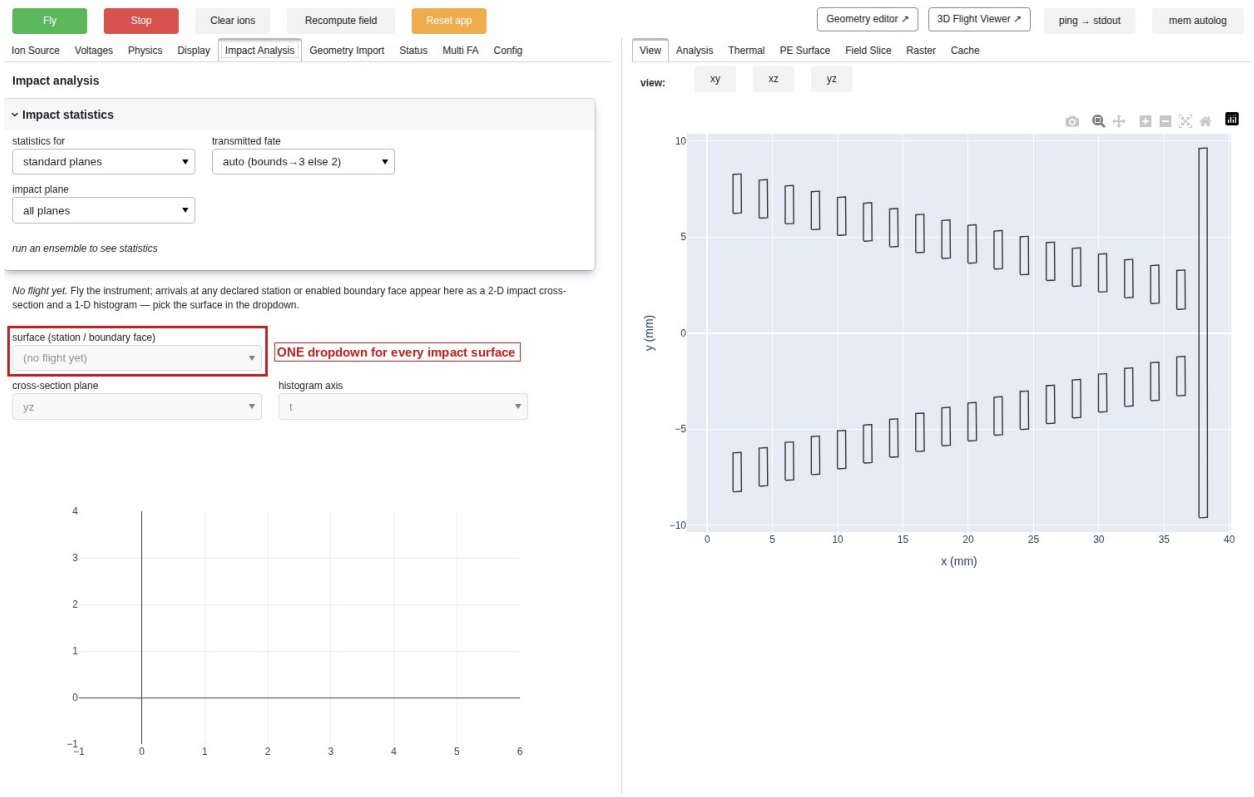


Display tab: what the view pane draws and how. The field overlay block at the top drives the shading and its contour lines; the contour slider is the single value authority and the off/on button flips it between 0 and the default of 16.

- **view plane (Select) —** Which 2-D plane to project the trajectories and field onto for display.
- field contour lines (0 = off) (IntSlider) — The single value authority for the contour count; the button beside it flips between 0 and the default 16.
- contours off/on (Button)
- paths drawn (0 = auto 25%) (IntInput) — How many ion paths to draw, evenly strided across the packet. 0 keeps the automatic policy: all up to 25, then 25% of the packet. Statistics always use every ion.
- shade field / potential (Checkbox)
- **shading type (Select) —** What the shading shows: instantaneous field magnitude |E|, the effective RF pseudopotential (PE) as a 2-D map, or the PE as a 3-D landscape.
- xz/yz shading slice (mm; auto at min) (FloatSlider)
- lock aspect ratio (equal x/y scale) (Checkbox)
- verbose build log (Checkbox)
- **trajectory style (Select) —** Draw each ion path as connected lines, individual recorded points, or both.
- line width (FloatSlider)
- opacity (FloatSlider)
- **color traces by (Select) —** Color each trajectory by its outcome (fate), by *m/z* (one color per mass, with a legend), a single color, or a recorded channel.
- trace color (ColorPicker)
- plot every Nth point (display only) (IntSlider)
- impact marker size (FloatSlider)

- **impact marker symbol (Select) —** Marker shape drawn where an ion terminates on metal or a bounding plane.
- electrode fills (Checkbox)
- electrode labels (Checkbox)
- electrode color (ColorPicker)
- electrode alpha (FloatSlider)
- Show field (no ions) (Button)

Impact Analysis

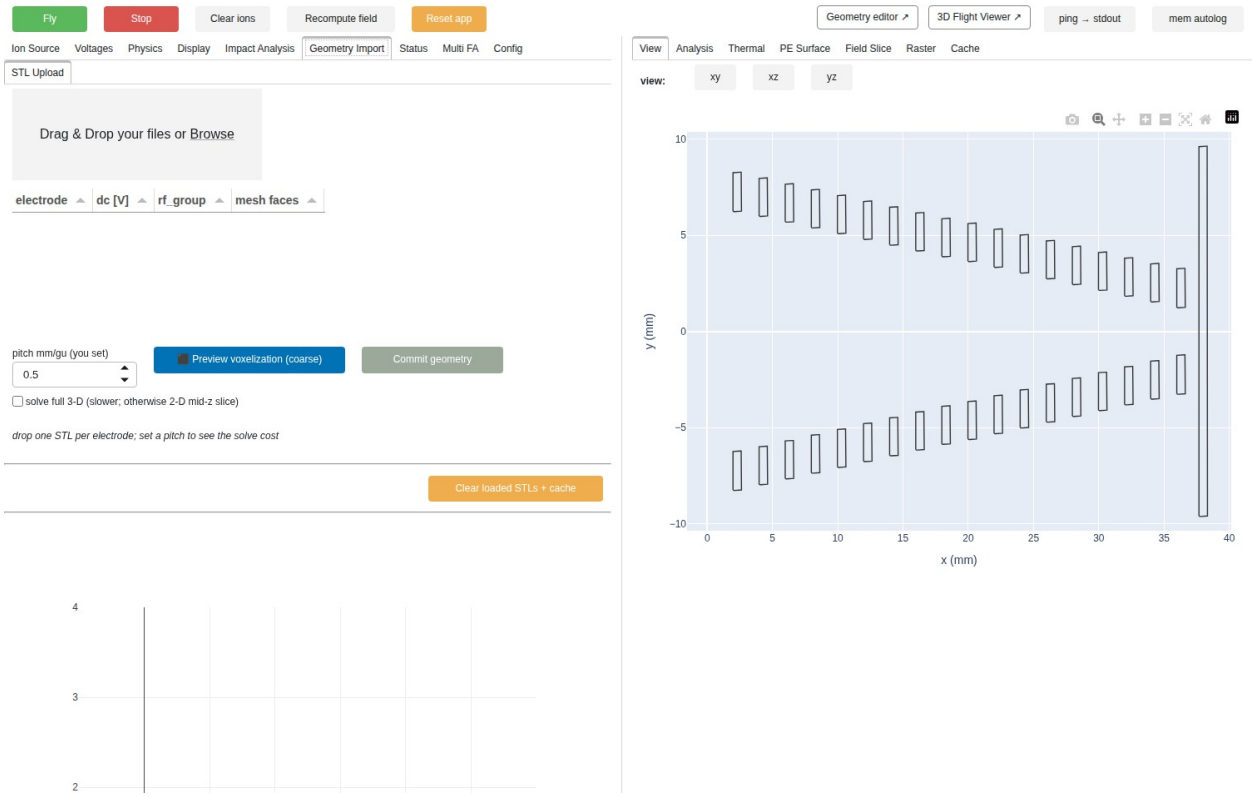


Impact Analysis tab before any flight: the surface dropdown enables once arrivals exist. Stations and enabled boundary faces share one stream and one naming authority (enabled_planes), so the dropdown and the stats table can never disagree. Single-stage flights and assemblies both feed it: station crossings from the kernels, assembly detections in world coordinates.

Impact analysis

- statistics for (Select)
- transmitted fate (Select)
- impact plane (Select)
- surface (station / boundary face) (Select)
- cross-section plane (Select)
- histogram axis (Select)

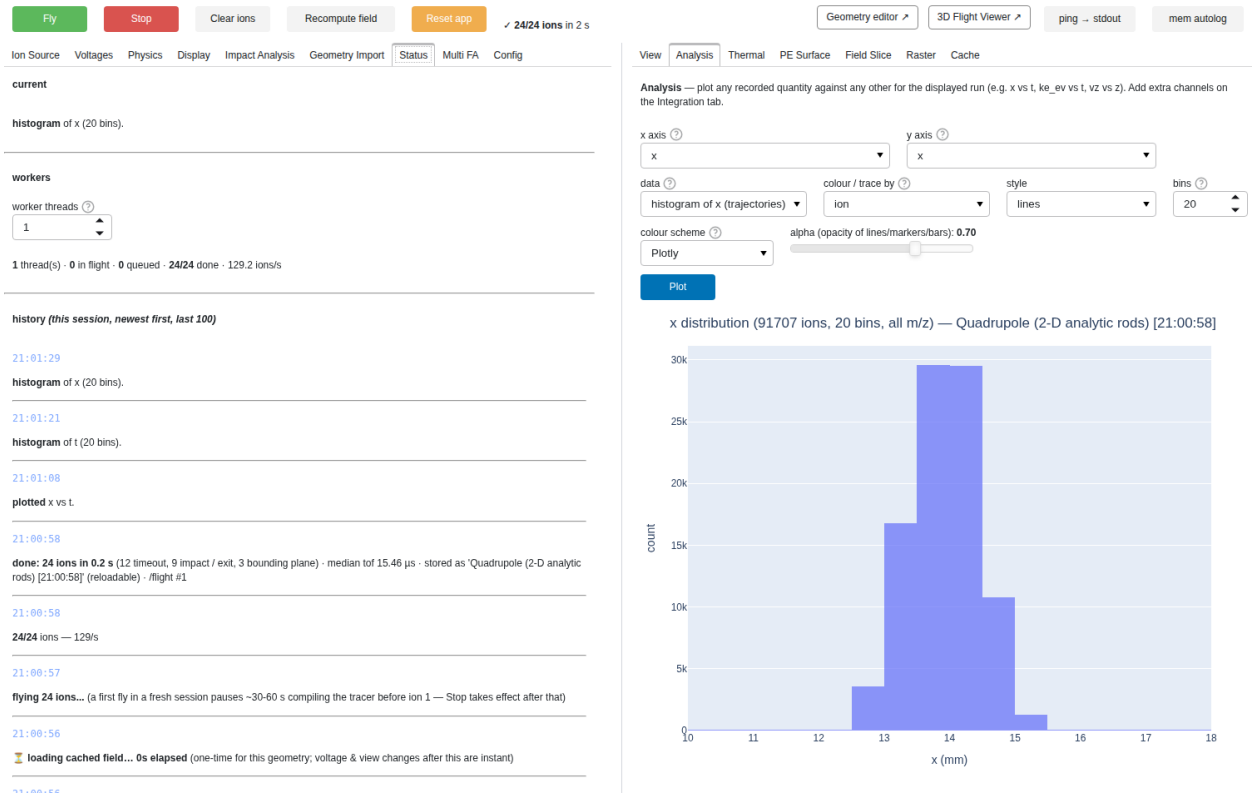
Geometry Import



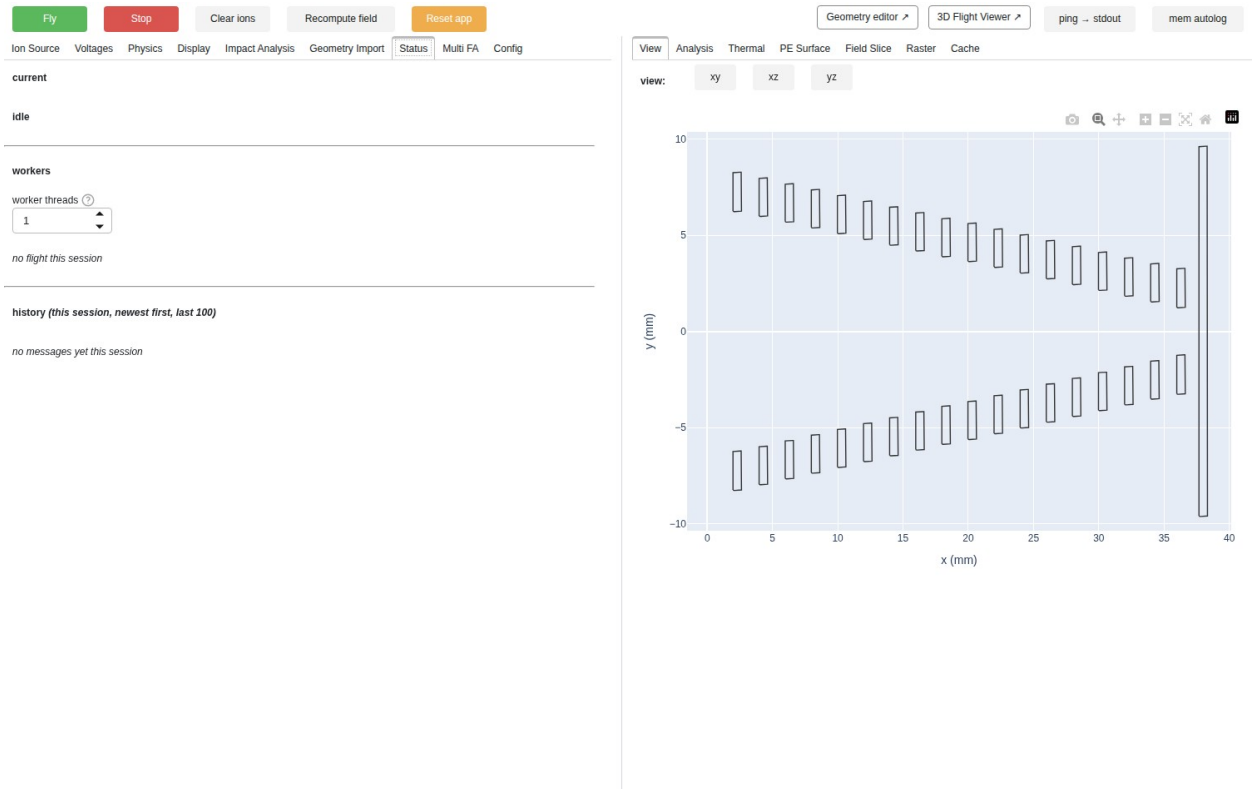
Geometry Import tab: bring electrodes in from JSON or STL.

- pitch mm/gu (you set) (FloatInput)
- Preview voxelization (coarse) (Button)
- Commit geometry (Button)
- solve full 3-D (slower; otherwise 2-D mid-z slice) (Checkbox)
- Clear loaded STLs + cache (Button)

Status



Status tab after flying Quadrupole (2D, analytic): the run log names the solve, the fly, and the fates.



Status tab: the run log.

The record-volume guard's verdicts land here (and on the fly chip in the top bar): the exact quote, the refusal or warning, and the levers (section 11). Two diagnostics live here too: ping → stdout prints one line to the server console — if the UI feels wedged, a ping that appears means the kernel is alive and

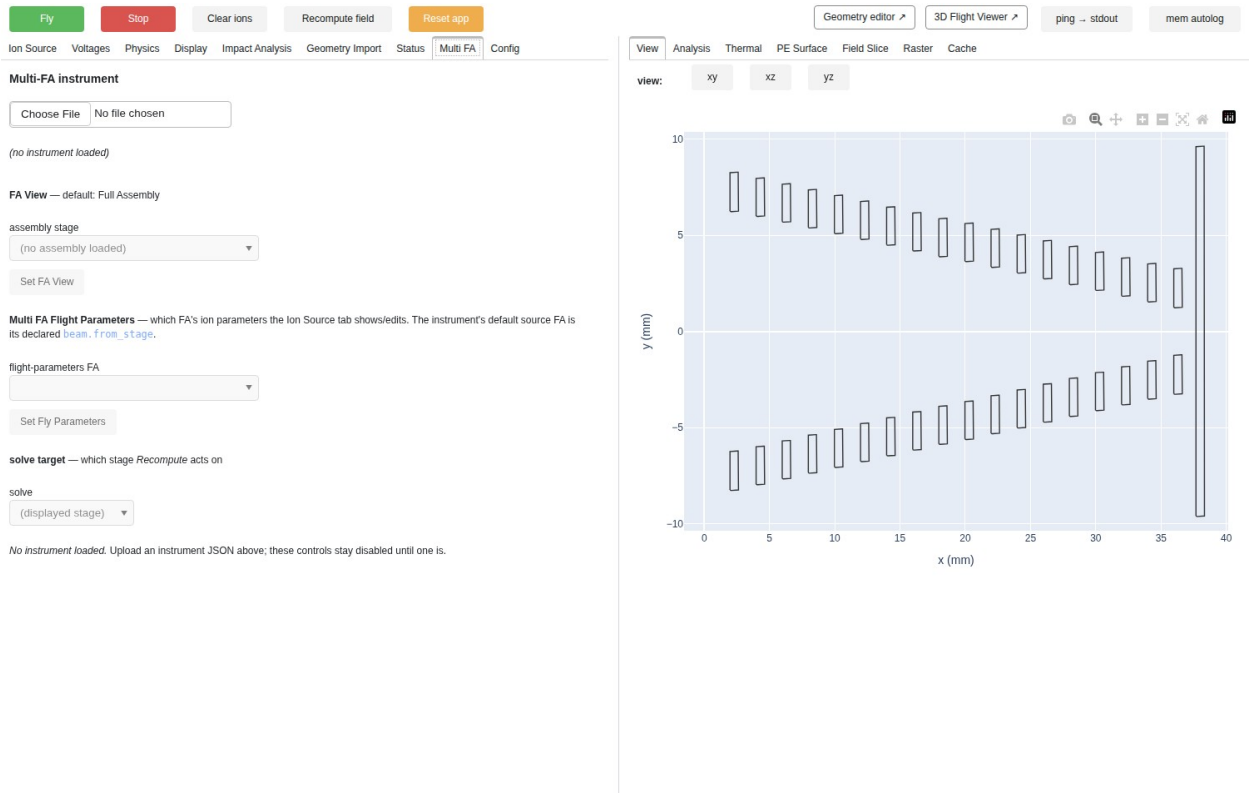
the browser tab is the problem — and mem autolog logs the process's memory use to stdout every 30 s, which survives a wedged interface.

current

idle

history *(this session, newest first, last 100)

Multi FA



Multi FA tab: staged assemblies -- each stage solves in its own grid and hands the ion across a declared seam.

Multi-FA instrument

FA View — default: Full Assembly**

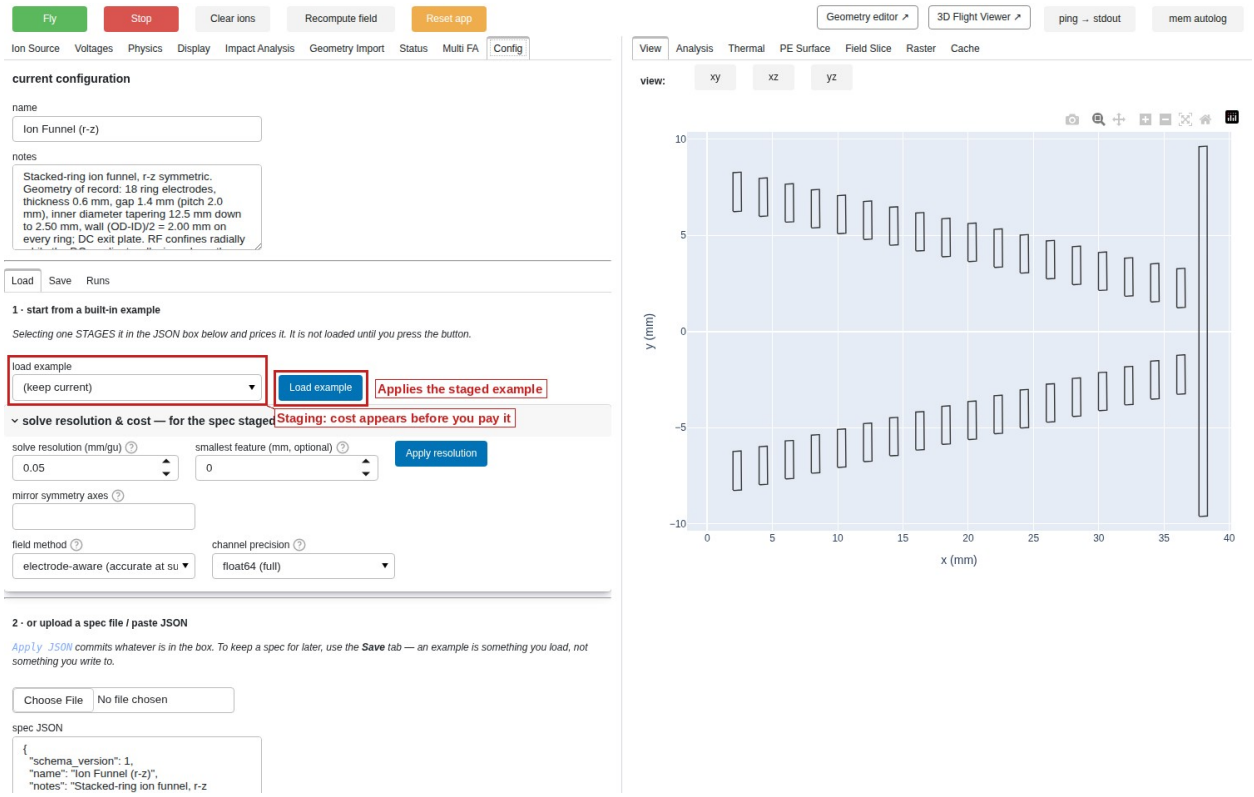
- assembly stage (Select)
- Set FA View (Button)
- flight-parameters FA (Select)
- Set Fly Parameters (Button)

solve target — which stage *Recompute* acts on**

- solve (Select)

Fly assembly (top bar) flies the loaded instrument: stages solve or cache-hit, the packet is generated from the declared beam, and progress streams to the status line from a worker thread. The result reads detected / flown with named reasons for the rest (section 10); the record guard prices the retained trajectory subset before any ion flies.

Config



Config tab: framework version, the examples path, and the example loader. Selecting an example STAGES it and shows its cost BEFORE you commit; Load example applies it.

current configuration

- name (TextInput)
- notes (TextAreaInput)
- load example (Select)
- Load example (Button)
- **solve resolution (mm/gu) (FloatInput) —** Grid pitch for the field solve. Smaller = finer = costlier (nodes scale as pitch^-3 in 3-D).
- **smallest feature (mm, optional) (FloatInput) —** The narrowest gap/slot you care about. Used ONLY to warn when the pitch cannot resolve it (<~3 cells).
- Apply resolution (Button)
- **mirror symmetry axes (MultiChoice) —** Reflecting-boundary symmetry planes. Each axis listed means the stored grid is HALF the domain (mirror plane at the low face) — the solver halves the work per axis and the field unfolds to the full instrument. Declare an axis ONLY when the geometry is truly symmetric about that plane and the electrodes sit against it. Changing this re-solves.
- **field method (Select) —** How E is differenced from the potential. Electrode-aware corrects the field at metal surfaces (essential when ions ride close to electrodes, e.g. a surface-born TOF source). Plain gradient skips that pass — faster, and correct when ions stay off the surfaces (e.g. SLIM central-gap transport).
- **channel precision (Select) —** Stored precision of the drive-channel field stacks. float32 halves their memory (matters as grids grow); the gradient is always computed in float64, only storage changes.
- worker threads (IntInput) — Ion-flight threads for the next Fly. The default leaves two cores for the app; 1 is the sequential legacy path. Results are seed-identical at any thread count.

- max record RAM (GB) (IntInput) — Memory the trajectory record may use. The Fly refuses above this and warns at half of it, using the same quote the sizing readout shows. 0 = derive the ceiling from this machine's memory (section 11).
- spec JSON (TextAreaInput)
- Apply JSON (Button)

spec summary *(live)

- **save as (filename) (TextInput) —** Filename for the downloaded configuration. Include the .json extension (added if omitted).
- Download spec (FileDownload)
- instrument export filename (TextInput) — Filename for Download instrument; .json is appended if missing.
- Download instrument (FileDownload) — The loaded multi-stage instrument document (section 10).

stored runs (reloadable)

- stored runs (Select)
- Reload run into view (Button)
- Export displayed run (FileDownload)
- Download run (NPZ) (FileDownload)
- **Export view + run (HTML) (FileDownload) —** Self-contained HTML of the CURRENT view (field / PE surface + ion trajectories) with the full run metadata (geometry, voltages, gas, integration, fates) embedded.
- **Save solved field (Button) —** Save the current geometry's solved bases as one named .npz in the fields folder (self-validating; safe to rename or send).
- **Save trajectories (reloadable here) (Button) —** Save the ACTIVE run's ion paths to this app's store as a named .traj.npz; reload it with 'Load as run' or open it in notebook 05. Stays server-side (see the save path in the Status tab). For a file to take elsewhere, use 'Download run (NPZ)' above.
- saved fields (Select)
- **↻ (Button) —** Rescan the fields folder.
- **Load field (Button) —** Validate the selected field against the LOADED geometry and reinstate it into the cache. Refuses on any mismatch — a wrong-geometry field is silently wrong physics.
- **Open saved field (Button) —** One-file session open (PI 2026-08-06): reconstruct the geometry + operating point from the file's embedded spec, load it into the editor as if you had pasted the JSON, then validate and reinstate the bases. Refuses pre-bootstrap saves (no embedded spec) with the reason.
- **label (TextInput) —** Local display label for the selected field (stored in the registry; never rewrites the npz).
- **Relabel (Button) —** Apply the label to the selected field (local registry only).
- saved trajectories (Select)
- **Load as run (Button) —** Load the selected trajectory file as a stored run (overlays even across geometries, with a warning).
- Clear stored runs now (Button)
- Clear field cache (Button)

View (plot pane)

view:

- xy (Button)

- xz (Button)
- yz (Button)

Analysis (plot pane)

- **x axis (Select) —** Recorded channel for the horizontal axis.
- **y axis (Select) —** Recorded channel for the vertical axis.
- **data (Select) —** Plot every recorded point, one point per ion at its final state, or a histogram of the x-axis quantity over per-ion endpoints (y axis is ignored).
- **color / trace by (Select) —** ion: a separate color per ion (cycling palette). all together: one series. fate: color by outcome. *m/z*: one color per *m/z* value — each ion is colored by its OWN flown *m/z* (stable palette, sorted numerically, shown in the legend); works in all three plot modes.
- style (Select)
- **bins (IntInput) —** Histogram bin count. A good rule of thumb is about sqrt(number of ions); the default 20 suits ~50-400 ions. Only used in histogram mode.
- ***m/z* filter (MultiChoice) —** Empty = ALL *m/z* flown (default). Pick one or more values to plot only those ions. Options come from the displayed run's FLOWN ions (each ion's own recorded *m/z*), not from the editor — so a reloaded older run offers what it actually flew.
- **color scheme (Select) —** Palette for line/marker colors — applies to per-ion cycling, *m/z* coloring (stable assignment over sorted *m/z*), and the all-together series. Fate colors stay fixed: they are semantic (impact/alive/timeout), not aesthetic.
- alpha (opacity of lines/markers/bars) (FloatSlider)
- Plot (Button)

Thermal (plot pane)

- T_slope tail-fit percentile (FloatSlider)
- **bins (IntInput) —** Histogram bins for the velocity and energy distributions.
- Assess temperature (Button)
- ions (IntInput)
- cell size (mm) (FloatInput)
- hot threshold (eV; blank = 10 kT) (FloatInput)
- Map hot spots (Button)

PE Surface (plot pane)

- ion *m/z* (Da) (FloatInput)
- component (Select)
- plane (Select)
- slice position (mm; auto if at min) (FloatSlider)
- slice (mm, exact) (FloatInput)
- electrodes on the PE scale (Select)
- trust radius (cells from metal) (IntSlider)
- drape current trajectories (Checkbox)
- height (z aspect) (FloatSlider)

- PE transform (optional) (Select)
- PE transform factor / asinh knee (eV) (FloatInput)
- resolution (pts/axis, ↓ = faster) (IntSlider)
- Compute surface (Button)

Field Slice (plot pane)

- quantity (Select)
- ion *m/z* (Da) (FloatInput)
- plane (Select)
- slice position (mm; auto if at min) (FloatSlider)
- slice (mm, exact) (FloatInput)

heatmap

- heatmap (Checkbox)
- heatmap colors (Select)
- log scale (Checkbox)
- range min (blank=auto) (FloatInput)
- range max (blank=auto) (FloatInput)

contours

- contours (Checkbox)
- # contours (IntSlider)
- contour colors (Select)
- resolution (pts/axis, ↓ = faster) (IntSlider)
- lock aspect (true mm) (Checkbox)

view window (mm; blank = full domain)**

- x min (blank=full) (FloatInput)
- x max (FloatInput)
- y min (blank=full) (FloatInput)
- y max (FloatInput)
- reset view (Button)
- Compute slice (Button)
- CSV filename (TextInput)
- ⤓ export slice CSV (FileDownload)

Raster (plot pane)

- **Compute 3D View (Button) —** Render the labeled voxel grid the SOLVER consumes (route-agnostic: scene CSG, STL voxelization). Multi-axis: 3-D cloud + exact xy/xz/yz projections.
- show electrodes (Checkbox)
- show legend (Checkbox)

Cache (plot pane)

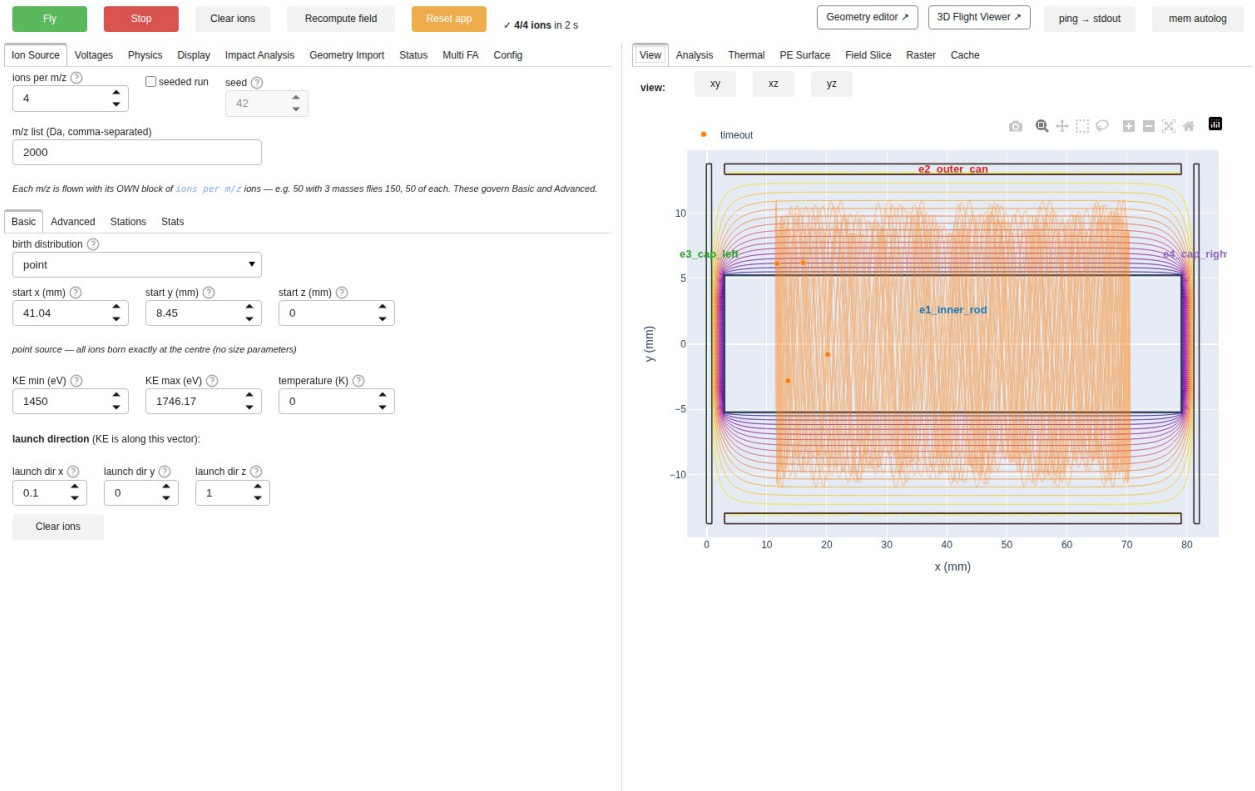
Cache location: ~/.ion_gym_cache by default (the ION_GYM_CACHE environment variable moves it); the pane lists each entry with its size on disk.**

- ↻ refresh (Button)
- clear ALL field cache (Button)
- entry to remove (Select)
- ✕ remove selected (Button)
- export to (TextInput)
- ⇩ export selected (field + spec json) (Button)

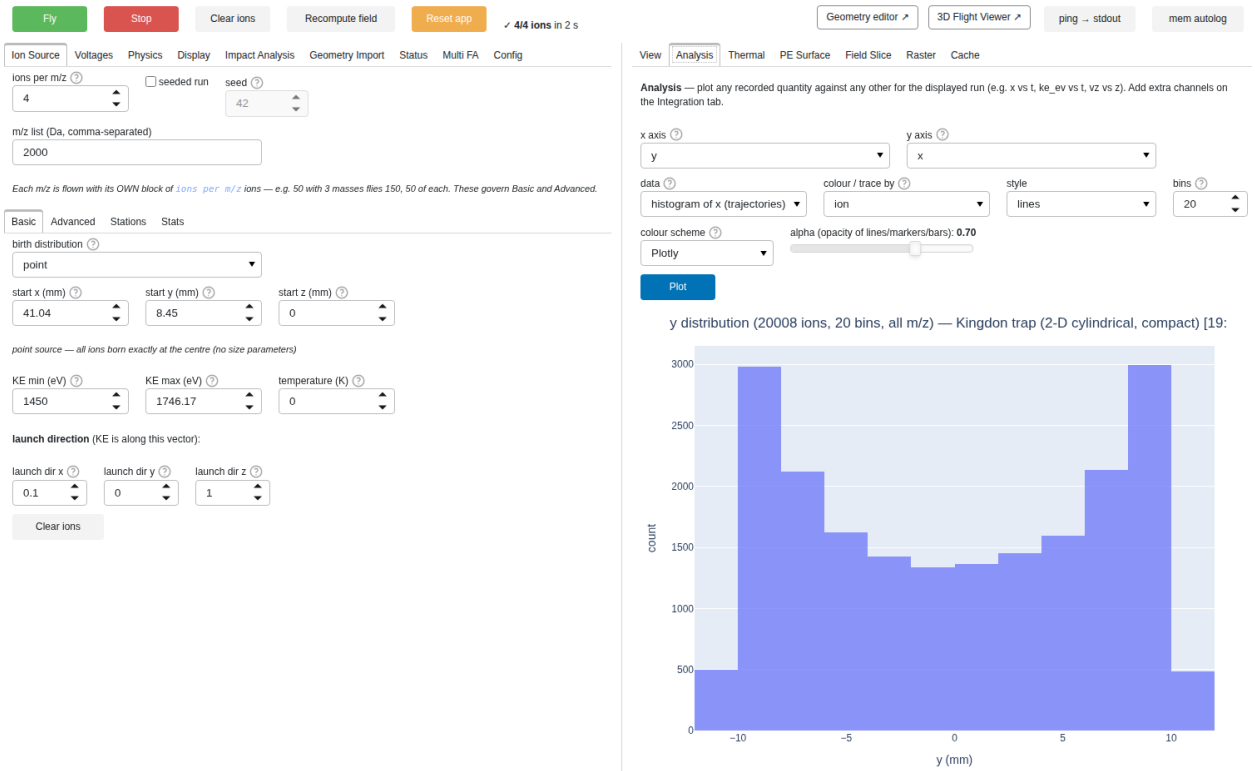
In-memory caches: (freed on 'clear ALL' or when the app restarts): - compose gradients: 0 geometry, 0.0 MB**

Worked example captures

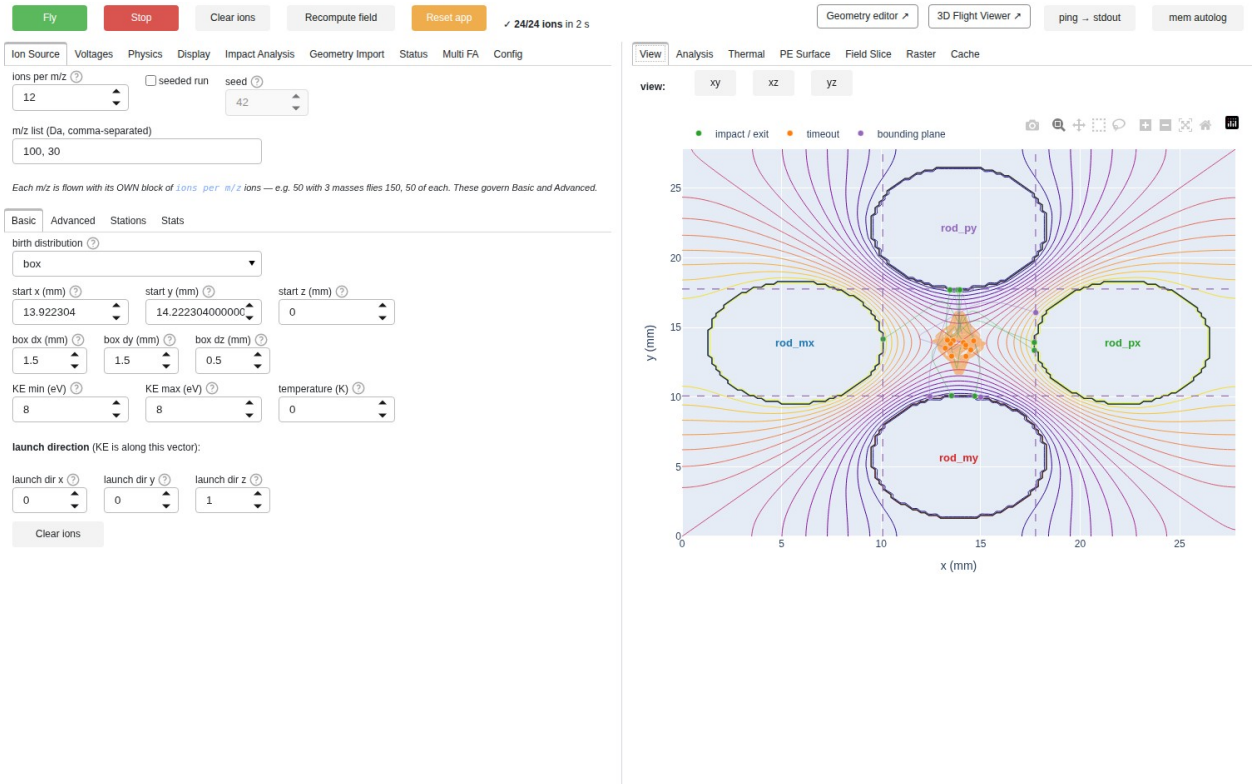
Each capture below is a real flight of a shipped example deck, driven through the dashboard by the screenshot pipeline (internal/tools/manual_build/capture_ui_screens.py) — nothing is mocked, and the set regenerates identically after any UI change by re-running the capture sessions.



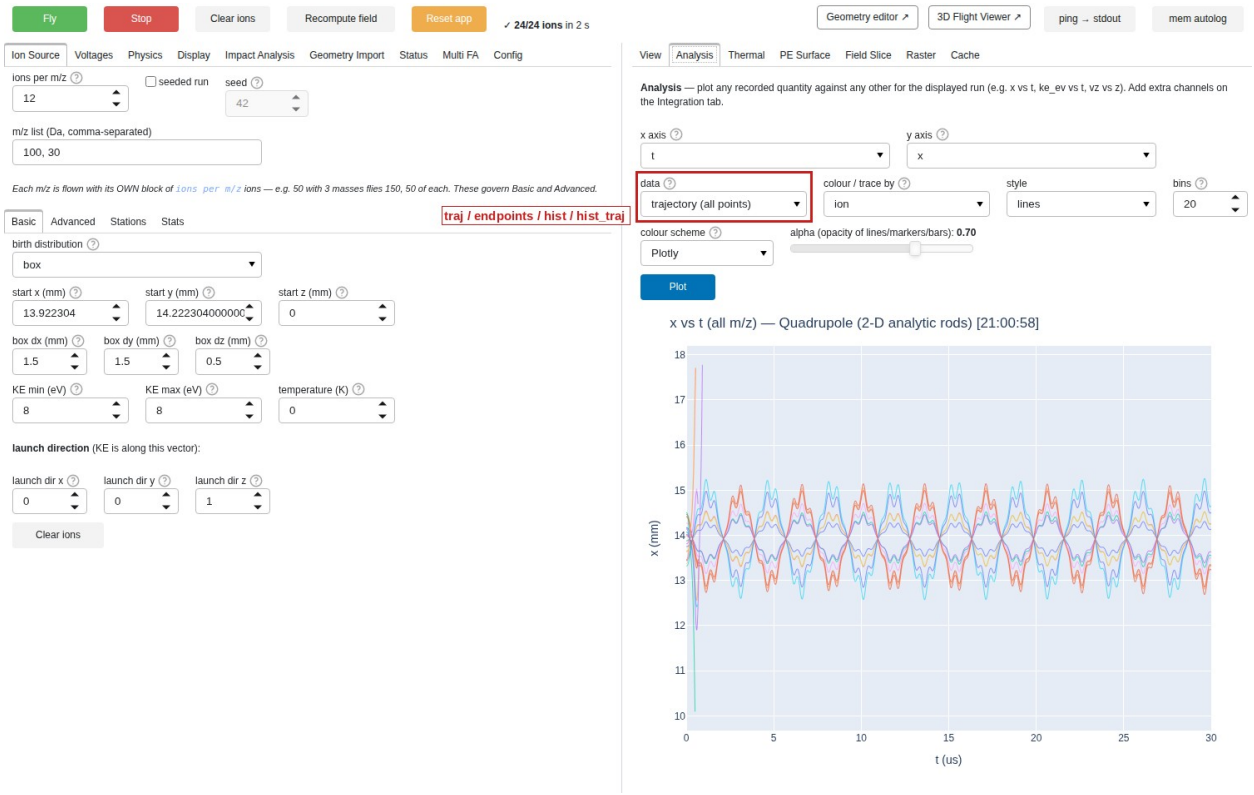
Kingdon Trap (r-z): orbital motion around the central wire (0.65 r / 0.76 z rescale verified).



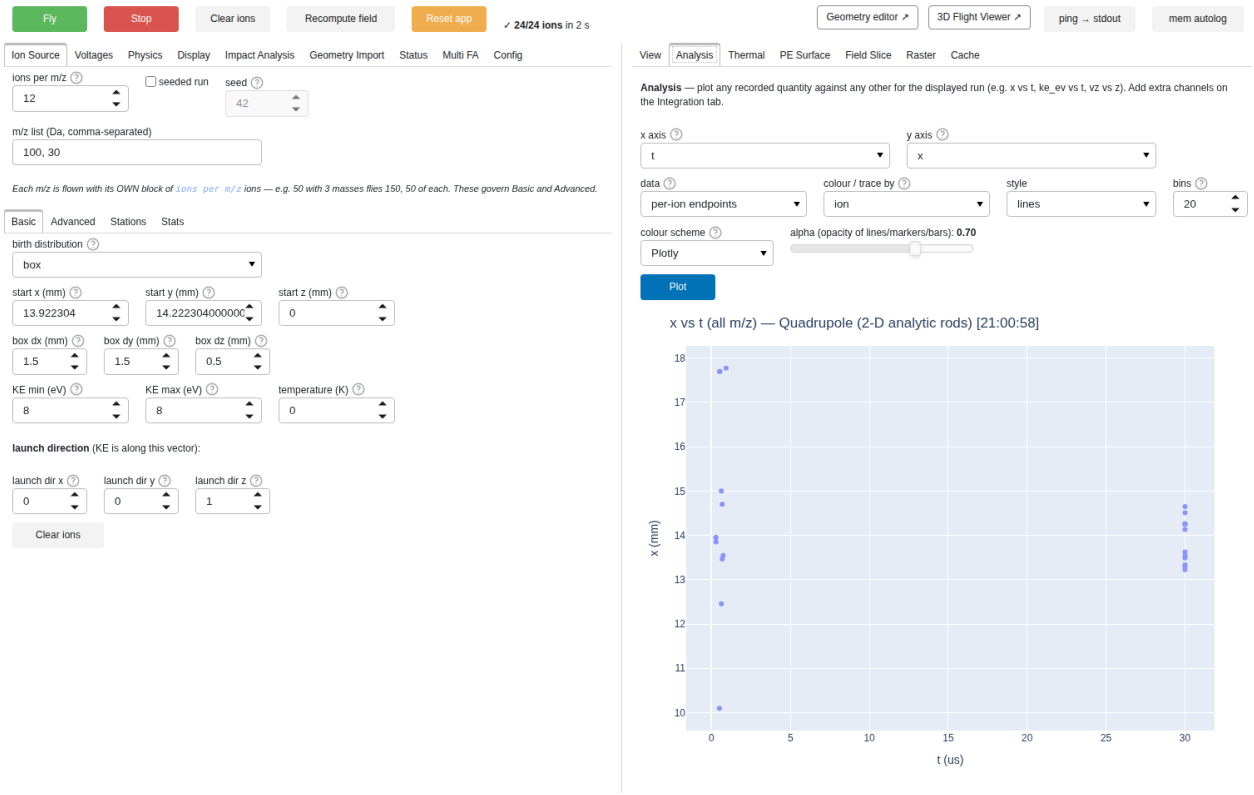
Kingdon Trap (r-z): hist_traj -- the trajectory-sample histogram of the radius (y in r-z) -- the mode shipped for orbit dwell structure; the pile-up sits at the radial turning points.



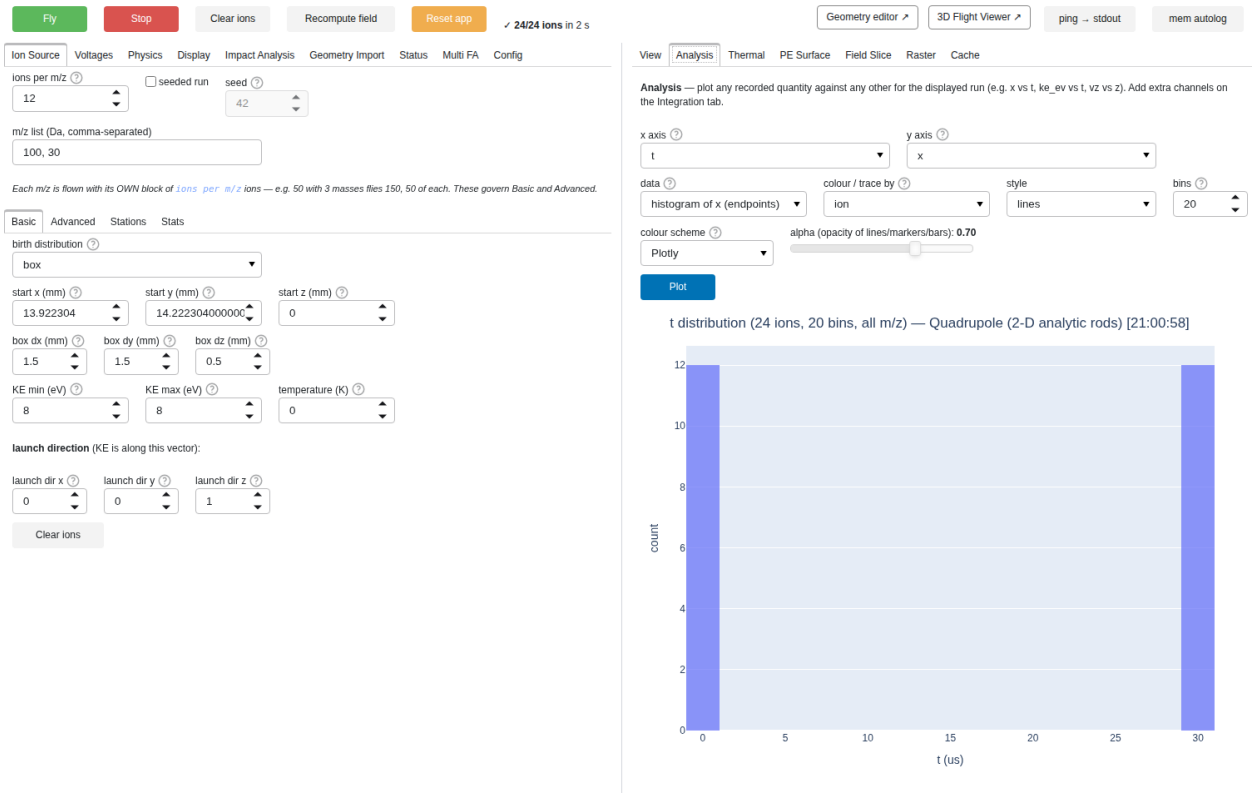
Quadrupole (2D, analytic): stable and ejected masses in one fly -- the analytic rod field the toolkit ships. View pane, xy.



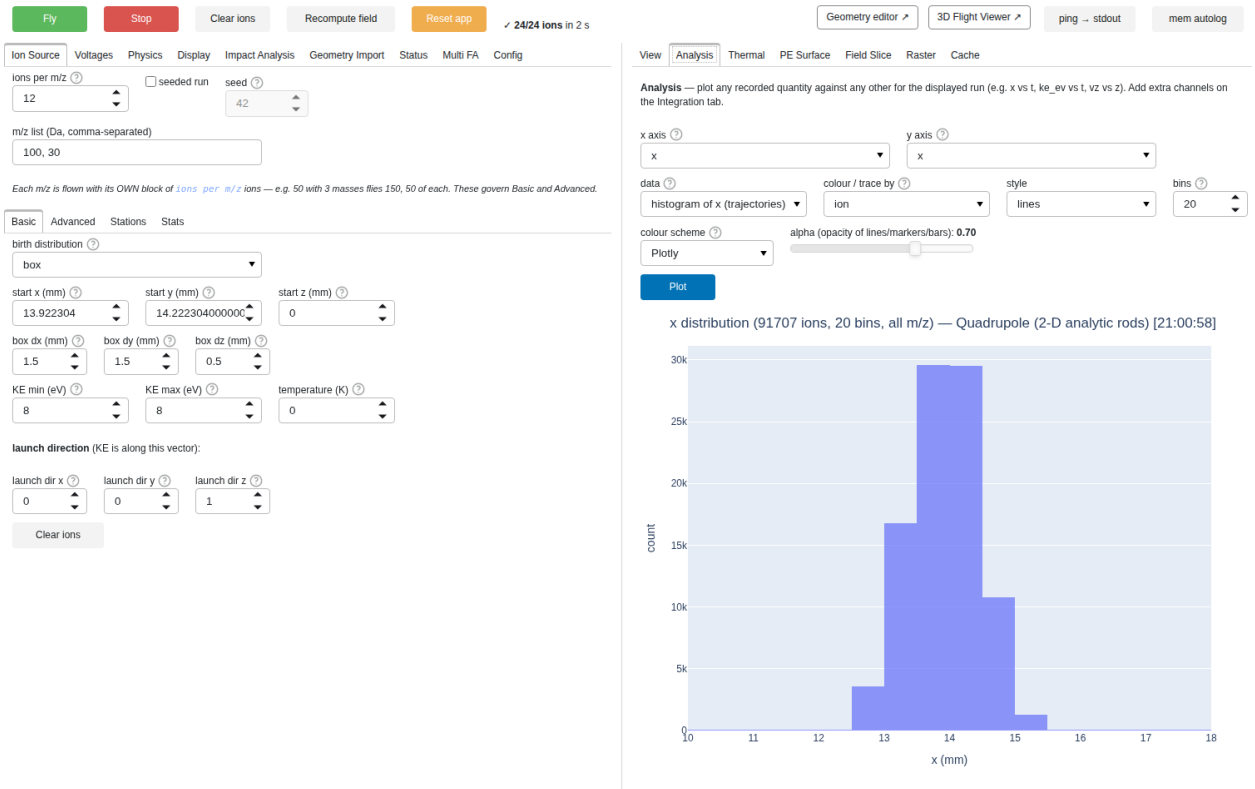
Quadrupole (2D, analytic): Analysis tab, trajectory mode (every recorded point of every ion).



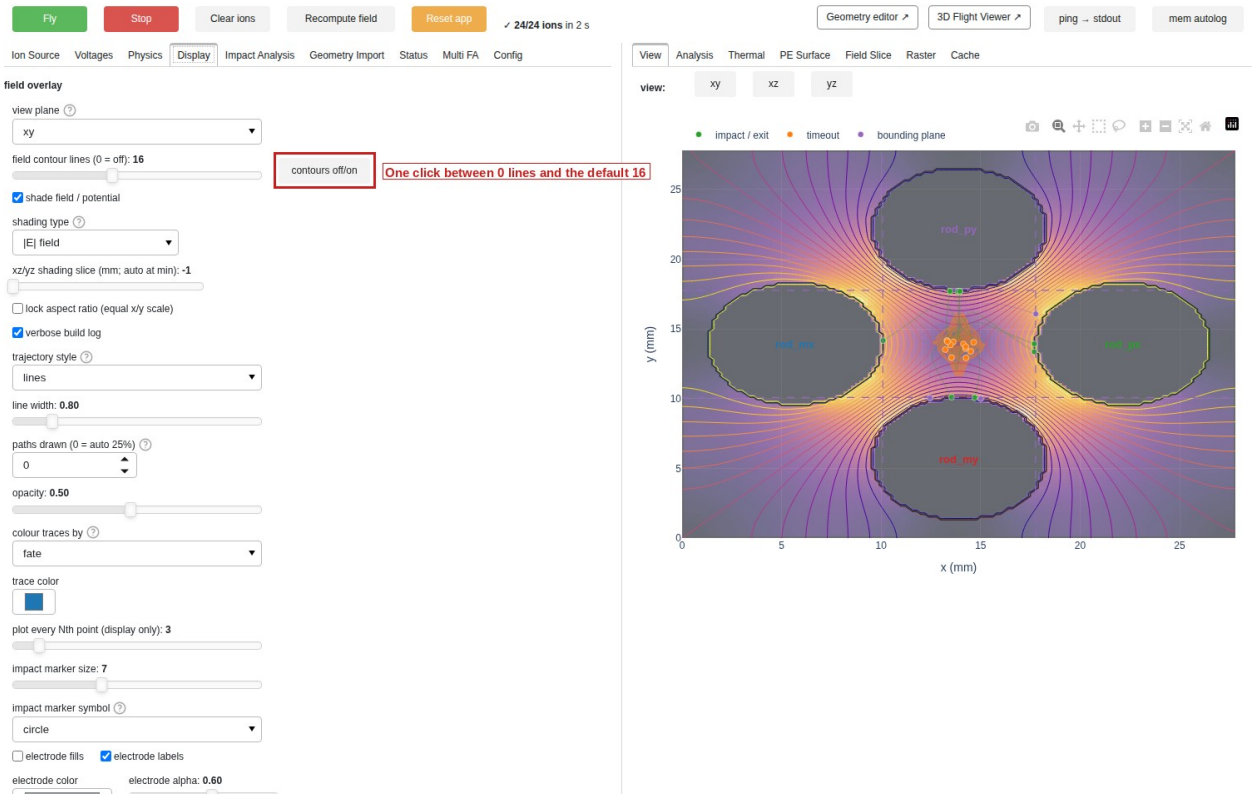
Quadrupole (2D, analytic): Analysis tab, per-ion endpoints.



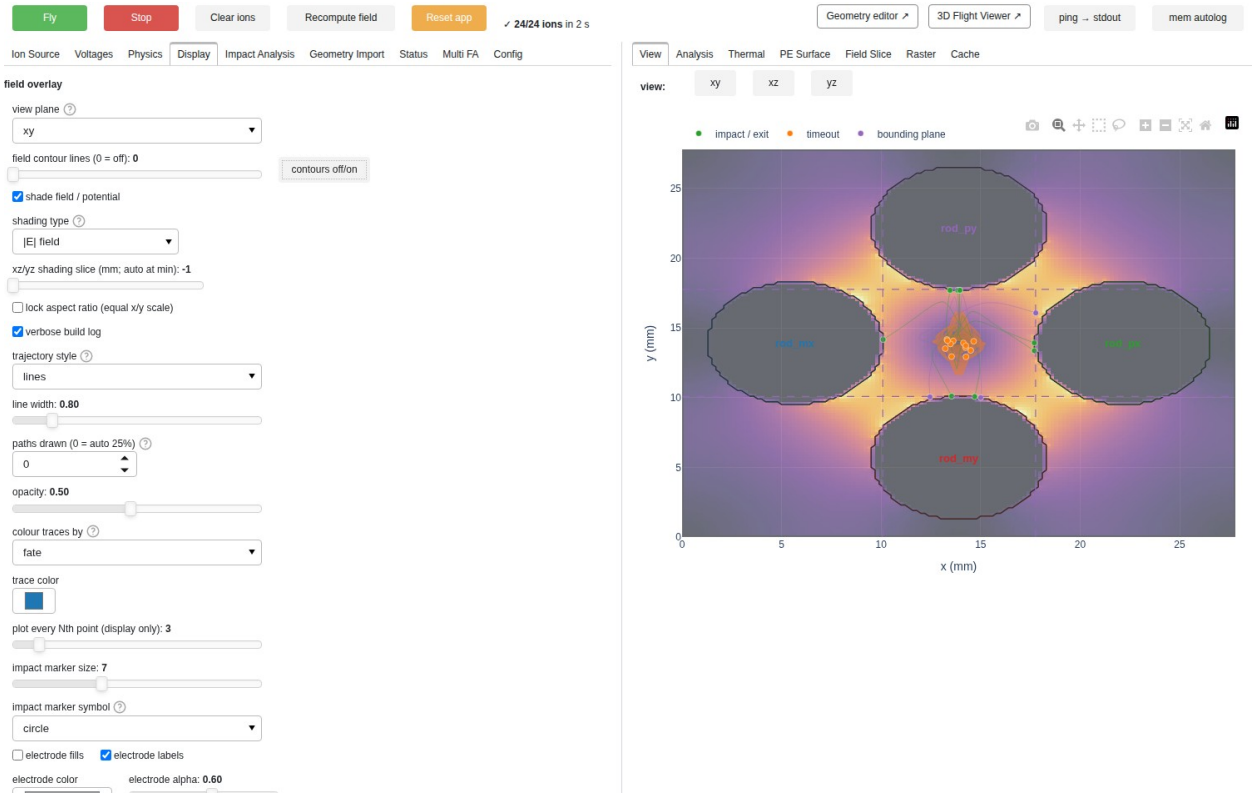
Quadrupole (2D, analytic): Analysis tab, histogram over per-ion endpoints.



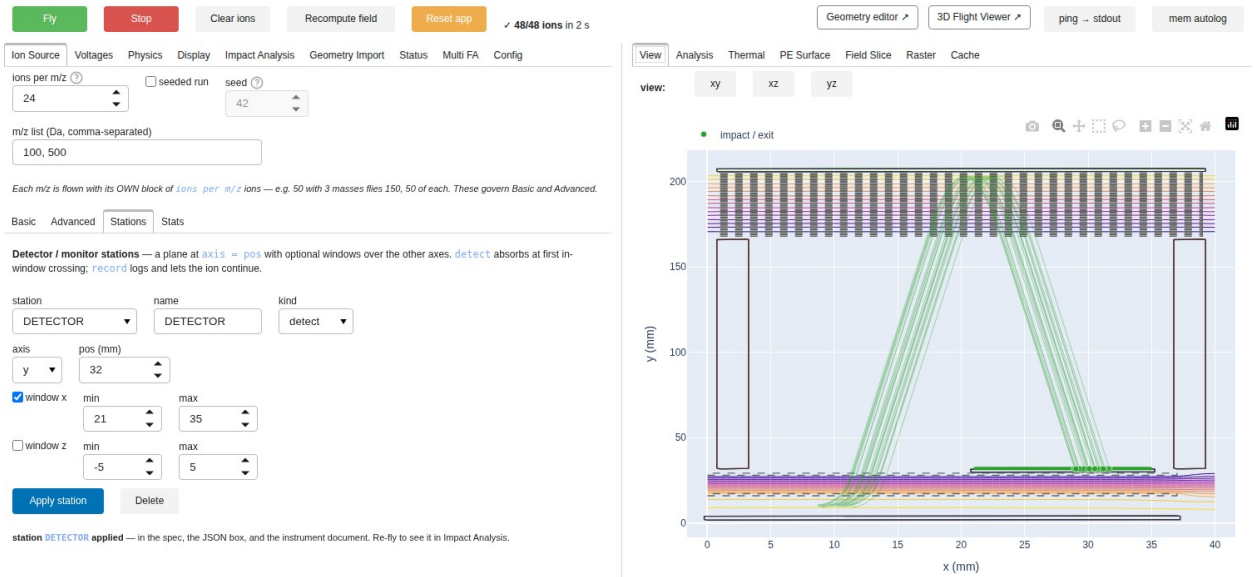
Quadrupole (2D, analytic): Analysis tab, histogram over EVERY recorded sample (hist_traj) of x -- residence-weighted, so where an ion dwells in the confinement plane is where the counts are.



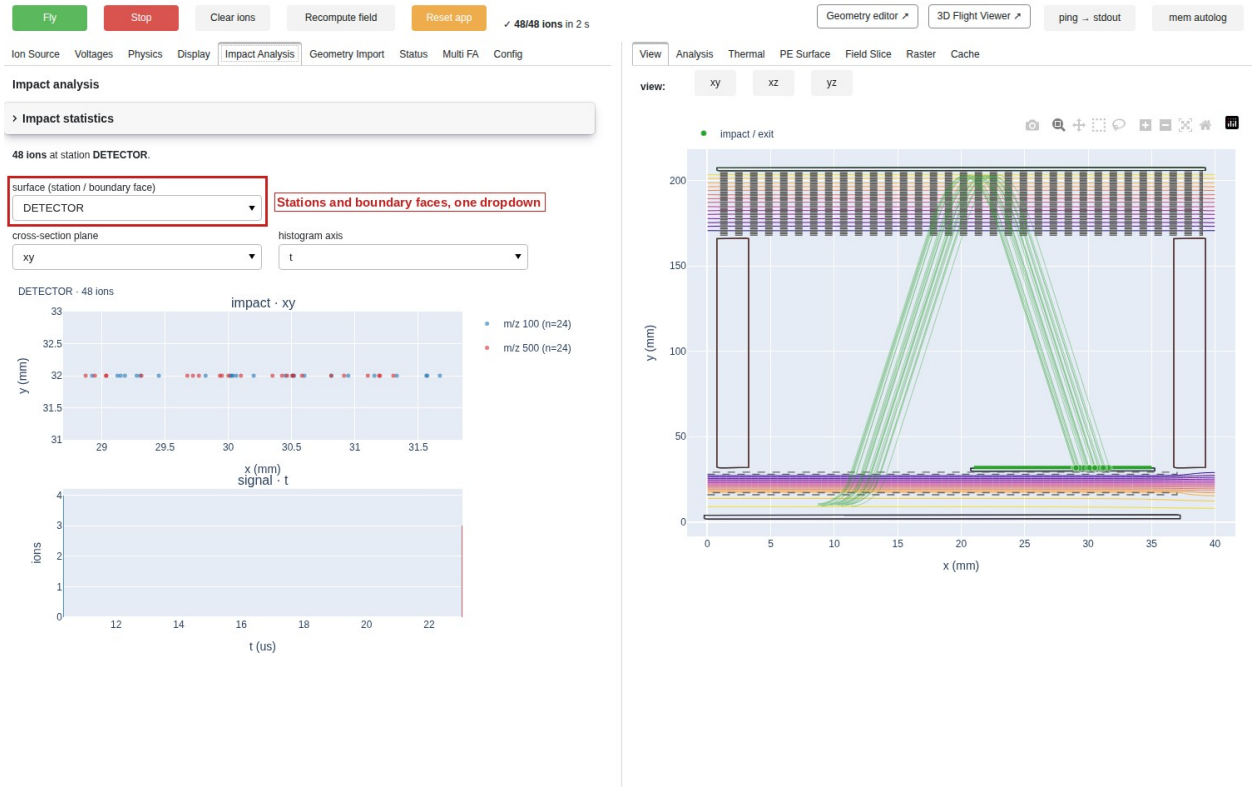
Quadrupole (2D, analytic): the field overlay ON over the same flight -- Display tab, 'shade field / potential' ticked, contour slider at its default of 16. The shading is the instantaneous |E| the ions flew through; the contour lines make the rod quadrupole's saddle structure legible under the trajectories.



Quadrupole (2D, analytic): the contours off/on button after one click -- the shading stays, the lines go, and the slider (the single value authority) now reads 0. A second click restores the default 16; dragging the slider sets any count in between.



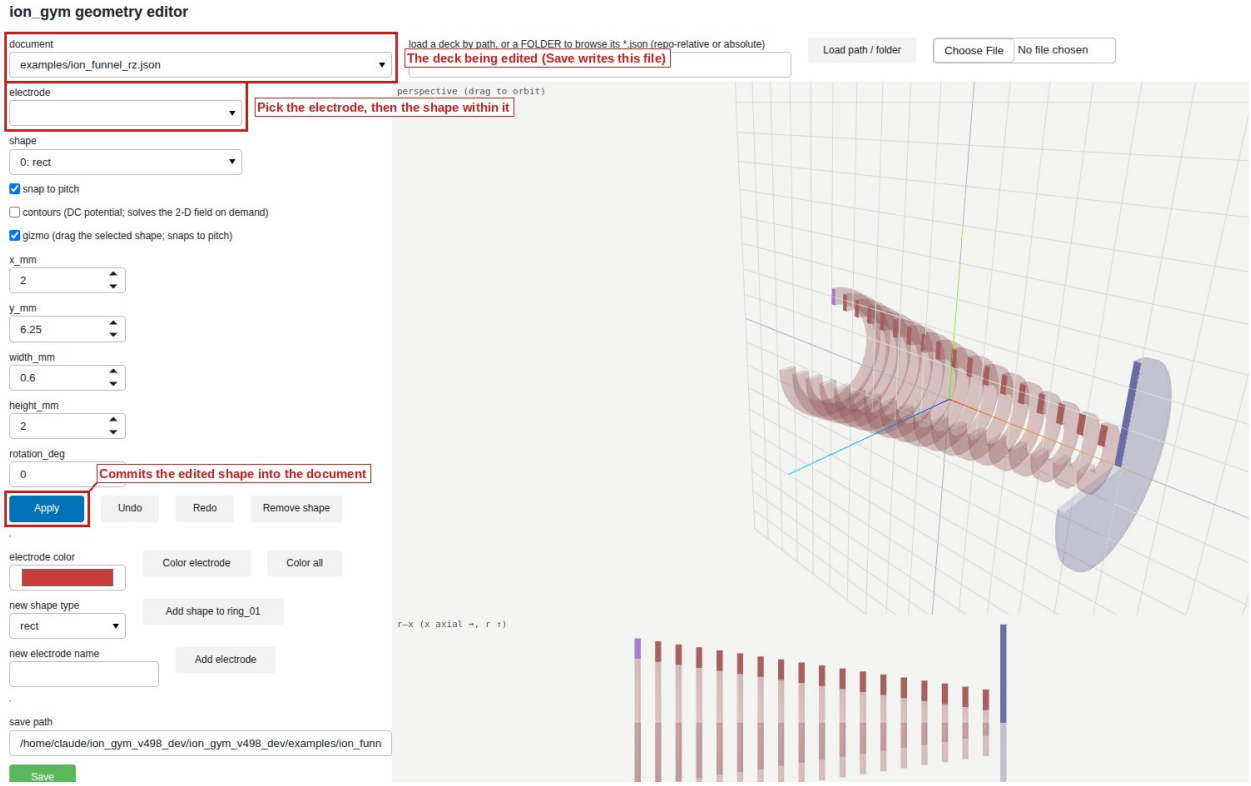
Reflectron oa-TOF (2D): orthogonal acceleration, the field-free drift, the ion mirror turnaround and the return to the detector plate, all in one View-pane capture of a real fly (m/z 100 and 500). Electrode labels are turned off here (Display tab checkbox) -- this stack is dense enough that the tags crowd the beam; your own session shows them by default.



Reflectron oa-TOF (2D): Impact Analysis at the DETECTOR station declared on the plate face (y = 32.0 mm, windowed to the 21-35 mm plate span). The impact-xy cross-section shows where the packet lands across the plate (this deck is planar, so z is identically zero); the signal-vs-t histogram of the same arrivals is the mass spectrum -- two masses, two peaks.

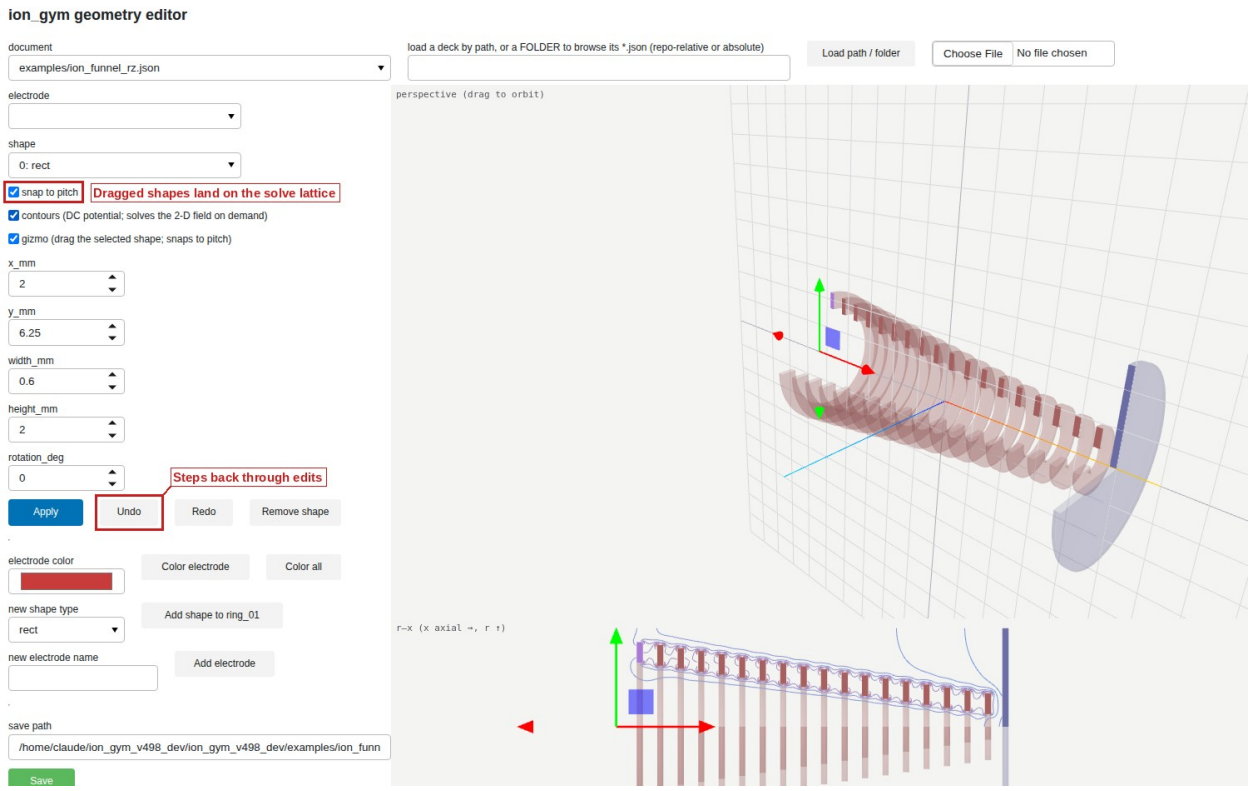
13.1 The geometry editor (/editor)

What it is. The dashboard's Geometry editor button opens /editor in a new tab, served by the same process. It edits the DECK: a shape is selected by electrode and index, its numbers are typed in millimetres or dragged with the gizmo, and Save writes the JSON back to the path shown at the bottom of the panel. The scene is the metal as the solver will rasterize it, not a sketch of it.



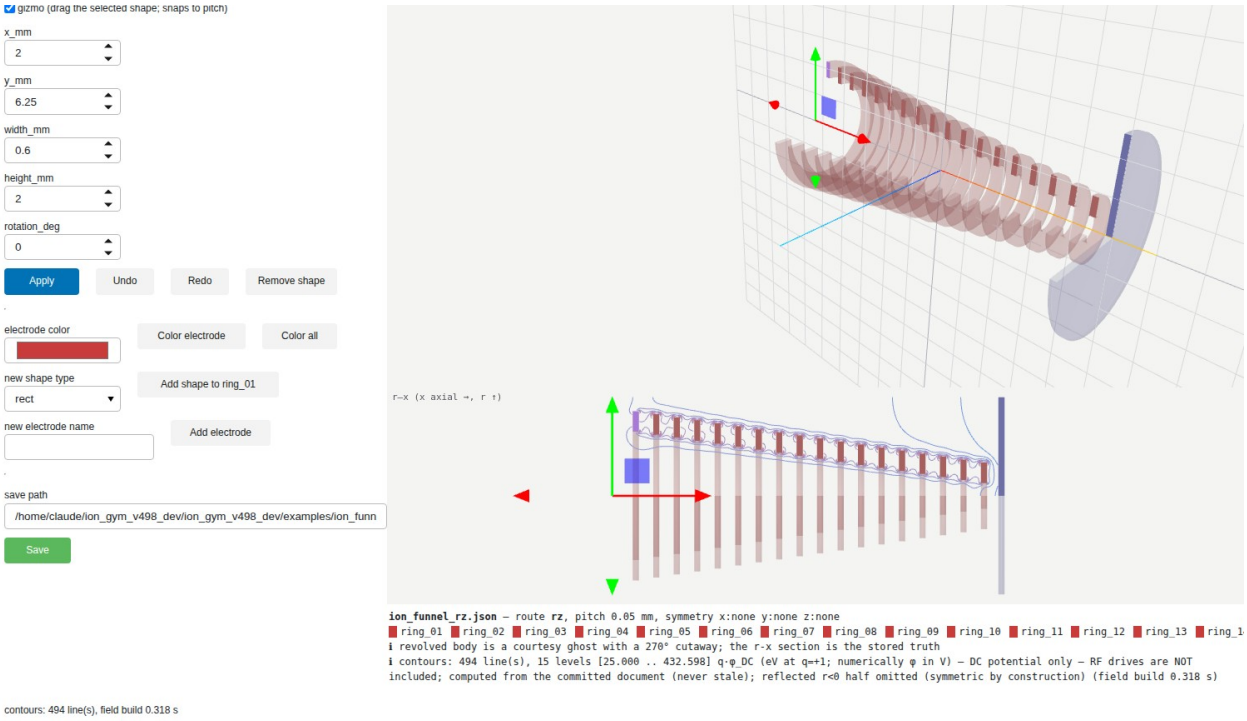
The geometry editor (/editor), opened on the ion funnel deck: the shape being edited is picked by electrode and index on the left, the perspective scene shows the metal as the solver will rasterize it, and the panel beneath it is the r-x section with the axial coordinate horizontal. Edits are applied to the document, not to a copy: Save writes the deck back to the path shown at the bottom.

Snap to pitch. With snapping on, a dragged shape lands on the solve lattice rather than between nodes. This matters for the same reason the lattice is declared rather than inferred: metal edges rasterize onto the grid, so an edge dropped half a cell off is a different instrument from the one you meant to draw. Undo and Redo step the document's edit history, and the editor refuses an edit it cannot represent rather than silently rounding it.



Snap-to-pitch and the drag gizmo: with 'snap to pitch' on, a dragged shape lands on the solve lattice rather than between nodes -- the editor's way of keeping metal edges where the rasterizer will actually put them. Undo and Redo step the document's edit history.

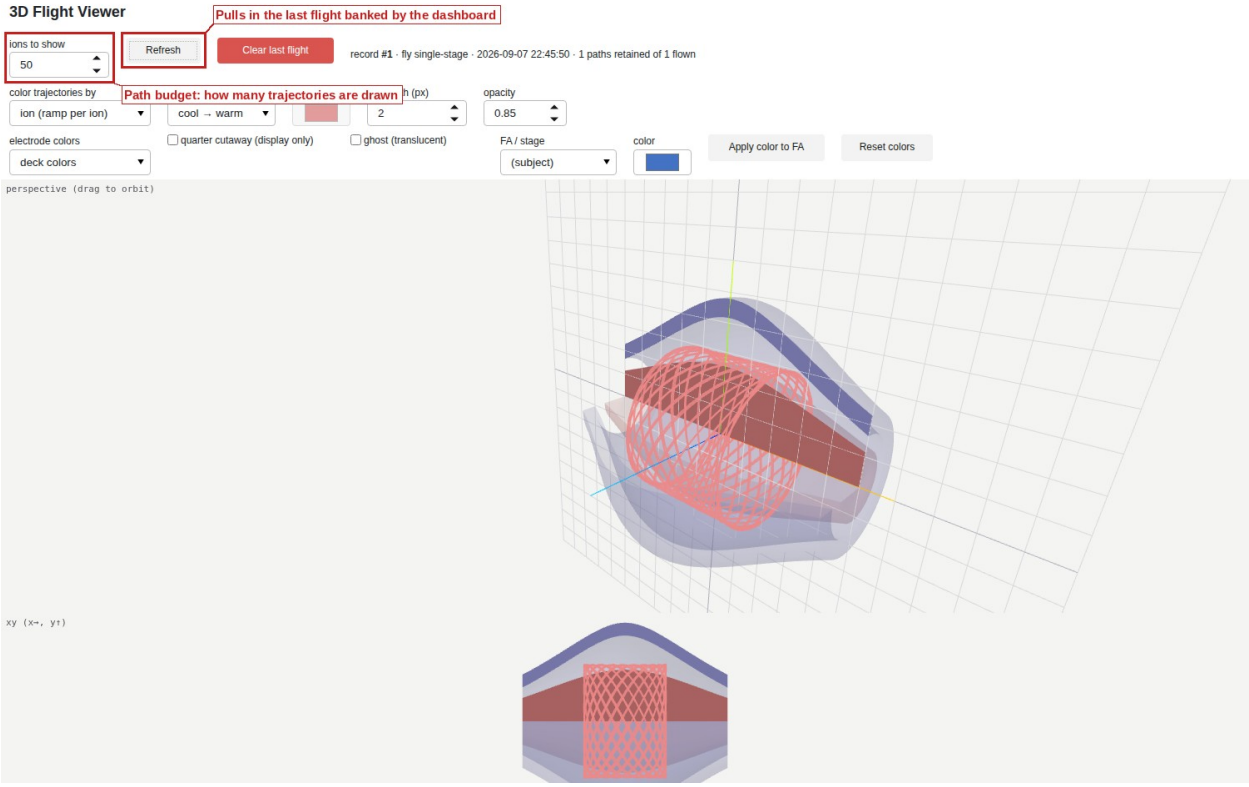
Contours in the editor. Ticking contours solves the two-dimensional DC field for the current document on demand and draws the equipotentials over the section, so the effect of moving an electrode is visible before the deck is ever flown. The editor states what it drew — for the ion funnel below, 494 contour lines across 15 levels, field build 0.318 s — and states equally plainly that these are DC potentials only, with RF drives excluded.



The editor's own contour overlay: ticking 'contours' solves the 2-D DC field for the deck on demand and draws the equipotentials over the section, so the effect of moving an electrode is visible in the editor before the deck is ever flown.

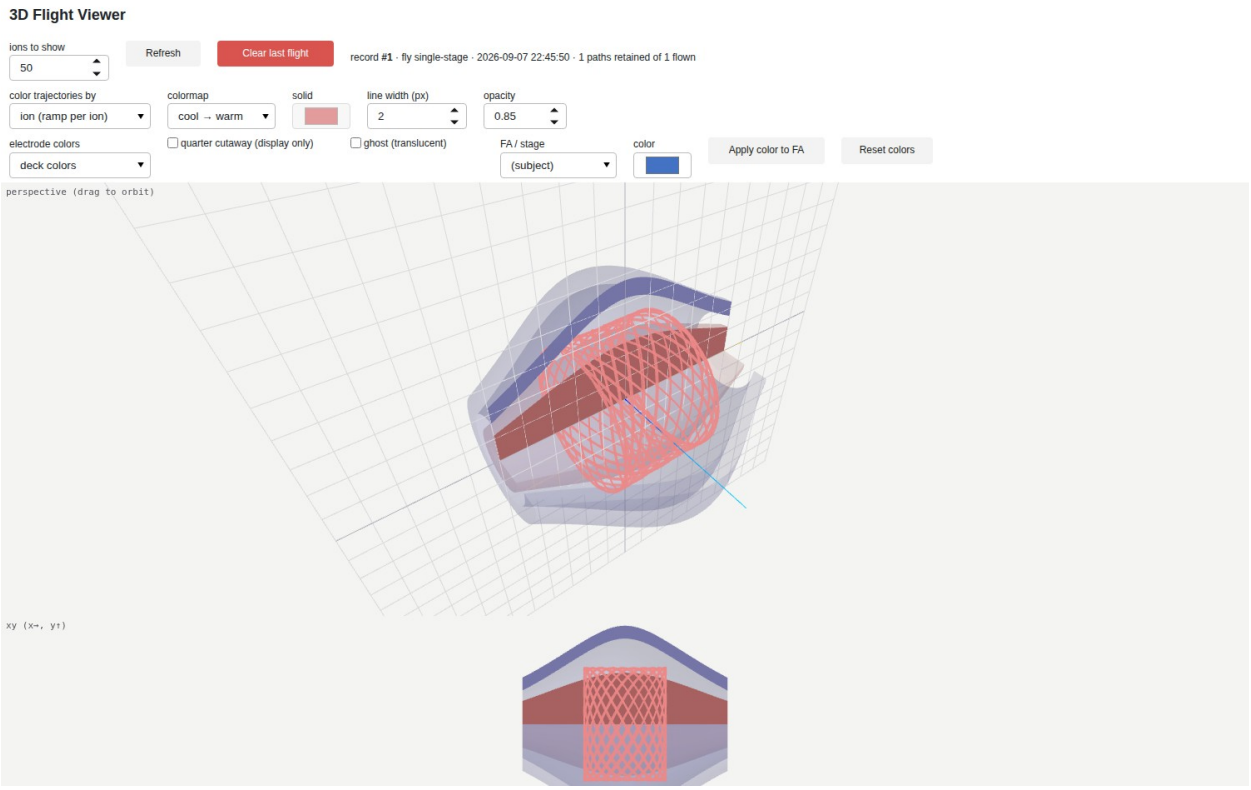
13.2 The 3-D Flight Viewer (/flight)

What it is. The 3D Flight Viewer button opens /flight, a read-only view of the LAST flight banked by the dashboard. Fly something, switch to the viewer, press Refresh, and the flown paths appear inside the instrument's electrodes in a scene you can orbit and zoom. The header states which record is being shown and how many paths were retained of those flown, so a thinned view can never be mistaken for the whole flight.



The 3-D Flight Viewer (/flight) on Orbitrap (r-z): the orbits flown in the dashboard, drawn in a rotatable scene with the trap's electrodes. The viewer reads the LAST banked flight -- fly in the dashboard, then press Refresh here. Trajectories are coloured per ion by default; drag to orbit, scroll to zoom, and the perspective is remembered per document across a refresh.

Why a second angle. A single projection of a three-dimensional scene hides an axis. The Orbitrap paths below are the same flight seen after dragging the camera through roughly a quarter turn: the first view shows the orbital motion around the spindle, the second shows the axial oscillation along it, and the instrument is only legible as both at once. The perspective is remembered per instrument, so a refresh returns to the view you left.



The same Orbitrap (r-z) flight after dragging the camera through roughly a quarter turn. A single projection of a 3-D scene hides an axis: here the second angle is what shows the orbits winding along the wire rather than simply circling it.

14. References

[1] D. A. Dahl, International Journal of Mass Spectrometry 200 (1–3), 3–25 (2000). doi:10.1016/S1387-3806(00)00305-5

[2] A. Brandt, “Multi-level adaptive solutions to boundary-value problems,” Mathematics of Computation 31 (138), 333–390 (1977). doi:10.2307/2006422

[3] D. M. Young, “Iterative methods for solving partial difference equations of elliptic type,” Transactions of the American Mathematical Society 76 (1), 92–111 (1954). doi:10.1090/S0002-9947-1954-0059635-7

[4] A. D. Appelhans and D. A. Dahl, International Journal of Mass Spectrometry 244 (1), 1–14 (2005).

[5] H. G. Dehmelt, “Radiofrequency spectroscopy of stored ions I: Storage,” Advances in Atomic and Molecular Physics 3, 53–72 (1967).

[6] D. Gerlich, “Inhomogeneous RF fields: a versatile tool for the study of processes with slow ions,” Advances in Chemical Physics 82, ch. 1 (1992).

[7] S. A. Shaffer, K. Tang, G. A. Anderson, D. C. Prior, H. R. Udseth and R. D. Smith, “A novel ion funnel for focusing ions at elevated pressure using electrospray ionization mass spectrometry,” Rapid Communications in Mass Spectrometry 11 (16), 1813–1817 (1997).

[8] W. C. Wiley and I. H. McLaren, “Time-of-flight mass spectrometer with improved resolution,” Review of Scientific Instruments 26 (12), 1150–1157 (1955). doi:10.1063/1.1715212

References [1], [4], [5] and [8] are the documented origins of methods implemented or ported in this toolkit and are recorded as such in the source. All entries above were verified against the publisher record rather than reproduced from memory.

Appendix A. The instrument specification, shape by shape

Section 3 lists the top-level blocks of a specification; this appendix goes one level down, to the part you actually author when you build an instrument: the electrodes and their shapes. Every fragment here is taken from a deck that ships in examples/ and loads through the same SimSpec.from_json a user's own file takes.

A.1 The skeleton

An electrode is a named conductor with a list of shapes, a DC value, and optional drive-group memberships. The geometry block holds the electrodes plus the grid declaration (extents, pitch, symmetry, and — for signed frames — origin_mm):

The geometry block below is complete and valid as written — it is the analytical quadrupole assembled in full in A.5, which builds, solves, and flies exactly as printed:

```
"geometry": {
 "width_mm": 21.8,
 "height_mm": 21.8,
 "depth_mm": 0.0,
 "mm_per_gu": 0.1,
 "symmetry": {
  "coords": "xyz",
  "planes": {
   "x": "none",
   "y": "none",
   "z": "none"
  }
 },
 "electrodes": [
  {
   "name": "rod_east",
   "dc": 0.0,
   "rf_groups": [
    "RFA"
   ],
   "shapes": [
    {
     "type": "ellipse",
     "cx_mm": 17.344,
     "cy_mm": 10.9,
     "rx_mm": 3.444,
     "ry_mm": 3.444
    }
   ]
  },
  {
   "name": "rod_west",
   "dc": 0.0,
   "rf_groups": [
    "RFA"
   ],
   "shapes": [
    {
     "type": "ellipse",
     "cx_mm": 4.456,
     "cy_mm": 10.9,
     "rx_mm": 3.444,
     "ry_mm": 3.444
    }
   ]
  },
```

```
    {
      "name": "rod_north",
      "dc": 0.0,
      "rf_groups": [
        "RFB"
      ],
      "shapes": [
        {
          "type": "ellipse",
          "cx_mm": 10.9,
          "cy_mm": 17.344,
          "rx_mm": 3.444,
          "ry_mm": 3.444
        }
      ]
    },
    {
      "name": "rod_south",
      "dc": 0.0,
      "rf_groups": [
        "RFB"
      ],
      "shapes": [
        {
          "type": "ellipse",
          "cx_mm": 10.9,
          "cy_mm": 4.456,
          "rx_mm": 3.444,
          "ry_mm": 3.444
        }
      ]
    }
  ],
  "rf_groups": [
    {
      "name": "RFA",
      "frequency_hz": 1 000 000.0,
      "amplitude_v": 30.0,
      "phase_deg": 0.0,
      "waveform": "sin"
    },
    {
      "name": "RFB",
      "frequency_hz": 1 000 000.0,
      "amplitude_v": 30.0,
      "phase_deg": 180.0,
      "waveform": "sin"
    }
  ]
}
```

coords is "xyz" for planar/3-D Cartesian builds and "rz" for axisymmetric ones (there x is the axis and y the radius). mm_per_gu is the grid pitch — the resolution of the physics (section 5) — and it is declared, never inferred.

A.2 The shape primitives

Five shape types exist: rect, ellipse, polygon, cutout, and group. All coordinates are millimeters in the deck frame. Any 2-D shape may carry an extrude descriptor giving it a range along one axis in a 3-D build; without one, a 3-D build treats it as a full-depth slab, and a 2-D build ignores extrusion entirely.

rect — an axis-aligned rectangle from its lower-left corner:

```
{"type": "rect", "x_mm": 21.0, "y_mm": 30.0,
 "width_mm": 14.0, "height_mm": 1.5}
```

This is the detector plate of the reflectron oa-TOF example: a 14 x 1.5 mm slab with its lower-left corner at (21, 30).

ellipse — center plus semi-axes (a circle when rx = ry), here with an extrusion making it a 50 mm rod:

```
{"type": "ellipse", "cx_mm": 3.7152, "cy_mm": 2.145,
 "rx_mm": 1.498, "ry_mm": 1.498,
 "extrude": {"axis": "z", "lo_mm": 0.0, "hi_mm": 50.0}}
```

This is rod_1 of the native 3-D hexapole example: a round rod of radius 1.498 mm extruded the full 50 mm along z. The cross-section plane is the pair of axes perpendicular to the extrude axis in cyclic order (z -> (x, y)), so cx/cy read as (x, y) here.

polygon — an arbitrary closed solid from its vertex list. The list is implicitly closed: the last vertex connects back to the first, so the first point is not repeated (repeating it is harmless but unnecessary):

```
{"type": "polygon",
 "points_mm": [[0.0, 0.0], [4.0, 0.0], [4.0, 1.0], [0.0, 2.5]]}
```

The rasterizer classifies polygons reflection-exactly — a polygon and its mirror image land on identical grid cells — which is what lets mirrored geometry be authored once and unfolded (A.4).

cutout — subtract a volume; the child shape is what gets removed:

```
{"type": "rect", "x_mm": 1.0, "y_mm": 168.0,
 "width_mm": 38.0, "height_mm": 0.9},
{"type": "cutout", "children": [
  {"type": "ellipse", "cx_mm": 20.0, "cy_mm": 168.45,
   "rx_mm": 5.0, "ry_mm": 0.45}]}
```

Shapes compose in declared order: solids union in, cutouts subtract out. A plate followed by an ellipse cutout is a plate with a beam aperture. A cutout that removes nothing on the grid is refused by name — a no-op subtraction is a declaration failure, not a smaller hole. group simply nests shapes so a repeated sub-assembly can be authored once.

A.3 Signed frames: origin_mm

By default the deck frame starts at (0, 0): node (0,0) of the grid sits at world (0, 0) and every coordinate is non-negative. origin_mm moves that anchor: it declares the world coordinate of grid node (0,0), so a device can be authored centered on the origin with signed coordinates — which is how you naturally measure a multipole. The hexapole example does exactly this:

```
"width_mm": 15.6, "height_mm": 15.6,
"origin_mm": [−7.8, −7.8],
...
{"type": "ellipse", "cx_mm": 3.7152, "cy_mm": 2.145, ...}
{"type": "ellipse", "cx_mm": −3.7152, "cy_mm": −2.145, ...}
```

The domain reads [-7.8, +7.8] mm in x and y, the six rods sit at +-R about (0, 0), the source is born at (0, 0), and the recorded trajectories come back in the same signed frame. origin_mm must sit on the pitch lattice (origin/mm_per_gu integer), and it is part of the geometry identity: two decks differing only in origin are different instruments to the solver cache.

A.4 A symmetry example

Symmetry is declared, never assumed (section 4). Declaring a mirror plane makes the solver store and relax only the non-negative half of that axis — half the memory, roughly half the time — while flight, display and recorded output run in the full, unfolded domain with the mirror plane at coordinate 0. The SLIM 3-D transport example declares one mirror:

```
"geometry": {
```

```
 "width_mm": 37.05, "height_mm": 12.35, "depth_mm": 1.6,
 "mm_per_gu": 0.05,
 "symmetry": {"coords": "xyz", "planes": {"z": "mirror"}},
 "electrodes": [
  {"name": "RF1", "shapes": [
   {"type": "rect", "x_mm": 3.277, "y_mm": 0.0,
    "width_mm": 0.406, "height_mm": 0.2}, ... ]},
 ...
}
```

On the mirrored axis, coordinates mean distance from the mirror plane: the stored half is z in [0, 0.8] and the flight domain reads z in [-0.8, +0.8] with the electrode pattern imaged across z = 0 — the two facing SLIM boards authored once. Shapes on a mirrored axis must not reach below 0 (the image supplies that side), and after every mirrored build the toolkit proves, rather than assumes, that the unfolded potentials are bit-symmetric about the declared plane. Because a mirror plane sits at 0 by construction, origin_mm on a mirrored axis is refused — the two mechanisms answer different questions and compose only on distinct axes.

A.5 A fully defined shape set: the analytical quadrupole

Everything above assembled into one complete instrument: four round rods, r0 = 3.0 mm, rod radius 1.148 r0 (the dodecapole-minimizing ratio), opposite pairs driven in antiphase at 1 MHz and 30 V. The domain is snapped to the lattice (21.8 mm = 218 cells at 0.1 mm/gu; the remainder of the physical envelope is absorbed at the outer walls, never at a symmetry plane). This deck is reproduced verbatim from a validated file: it passes validate(), builds, and a cold ion launched 0.35 mm off axis is held for the full 160 microsecond flight, oscillating about the center with the expected secular motion.

```
{
 "schema_version": 1,
 "name": "Analytical quadrupole (planar cross-section)",
  "notes": "Four round rods, r0 = 3.0 mm, r_rod = 1.148 r0 (dodecapole-minimizing ratio), opposite pairs driven in antiphase. Domain snapped to the lattice: 21.8 mm = 218 cells at 0.1 mm/gu.",
 "geometry": {
  "width_mm": 21.8,
  "height_mm": 21.8,
  "depth_mm": 0.0,
  "mm_per_gu": 0.1,
  "symmetry": {
   "coords": "xyz",
   "planes": {
    "x": "none",
    "y": "none",
    "z": "none"
   }
  },
  "electrodes": [
   {
    "name": "rod_east",
    "dc": 0.0,
    "rf_groups": [
     "RFA"
    ],
    "shapes": [
     {
      "type": "ellipse",
      "cx_mm": 17.344,
      "cy_mm": 10.9,
      "rx_mm": 3.444,
      "ry_mm": 3.444
     }
    ]
   },
   {
```

```
    "name": "rod_west",
    "dc": 0.0,
    "rf_groups": [
      "RFA"
    ],
    "shapes": [
      {
        "type": "ellipse",
        "cx_mm": 4.456,
        "cy_mm": 10.9,
        "rx_mm": 3.444,
        "ry_mm": 3.444
      }
    ]
  },
  {
    "name": "rod_north",
    "dc": 0.0,
    "rf_groups": [
      "RFB"
    ],
    "shapes": [
      {
        "type": "ellipse",
        "cx_mm": 10.9,
        "cy_mm": 17.344,
        "rx_mm": 3.444,
        "ry_mm": 3.444
      }
    ]
  },
  {
    "name": "rod_south",
    "dc": 0.0,
    "rf_groups": [
      "RFB"
    ],
    "shapes": [
      {
        "type": "ellipse",
        "cx_mm": 10.9,
        "cy_mm": 4.456,
        "rx_mm": 3.444,
        "ry_mm": 3.444
      }
    ]
  }
],
"rf_groups": [
  {
    "name": "RFA",
    "frequency_hz": 1 000 000.0,
    "amplitude_v": 30.0,
    "phase_deg": 0.0,
    "waveform": "sin"
  },
  {
    "name": "RFB",
    "frequency_hz": 1 000 000.0,
    "amplitude_v": 30.0,
    "phase_deg": 180.0,
    "waveform": "sin"
  }
```

```
    ]
  },
  "source": {
    "n_ions": 1,
    "distribution": "point",
    "x0_mm": 11.25,
    "y0_mm": 10.9,
    "mz_list": [
      100.0
    ],
    "charge": 1,
    "ke_lo": 0.0,
    "ke_hi": 0.0,
    "temperature_k": 0.0,
    "seed": 0,
    "tob_span_us": 0.0
  },
  "integration": {
    "dt_ns": 2.0,
    "t_max_us": 160.0,
    "rec_every": 25
  },
  "bounds": {},
  "collisions": {
    "enabled": false
  }
}
```


Paste it into a .json file and load it from the app or from SimSpec.from_json to fly it yourself.